

อัลกอริทึมการเรียงลำดับข้อมูล

เนื้อหาระดับมหาวิทยาลัยปี 1

หลักการ Bubble Sort Selection Sort Insertion Sort Merge Sort
Counting Sort

เอกสารสำหรับจัดทำชุดข้อมูลความรู้ของ AI Chatbot

จัดทำด้วยเนื้อหาเรียบเรียงใหม่จากเอกสารการสอนภาษาไทยของสถาบันอุดมศึกษา
โดยเน้นคำอธิบายที่ตรวจสอบได้ ตัวอย่างขนาดเล็ก Pseudocode และโค้ดภาษา Python

ระดับเนื้อหา: พื้นฐานโครงสร้างข้อมูลและอัลกอริทึมสำหรับนักศึกษาปริญญาตรีชั้นปีที่ 1

1 หลักการเรียงลำดับข้อมูล

บทนี้ปูพื้นฐานเรื่องข้อมูลนำเข้า ผลลัพธ์ ทิศทางการเรียง การเปรียบเทียบ การสลับ การเลื่อน และแนวคิดประสิทธิภาพที่ใช้ตลอดทั้งเอกสาร

ผลลัพธ์การเรียนรู้ของบท

- อธิบายแนวคิดด้วยภาษาของตนเองได้
- ติดตามขั้นตอนจากตัวอย่างข้อมูลขนาดเล็กได้
- อ่าน Pseudocode และโค้ดภาษา Python ได้
- ตรวจสอบผลลัพธ์และอธิบายข้อผิดพลาดได้

1.1 ความหมายและจุดประสงค์

ส่วนนี้อธิบายว่าการเรียงลำดับคืออะไร และเหตุใดจึงเป็นขั้นตอนสำคัญในการจัดการข้อมูล

1.1.1 ความหมายของการเรียงลำดับข้อมูล

เริ่มจากมองข้อมูลเป็นรายการที่มีสมาชิก และกำหนดความสัมพันธ์ก่อนหลังระหว่างสมาชิก

1.1.1.1 ความหมายของข้อมูลก่อนการเรียงลำดับ

ข้อมูลก่อนการเรียงลำดับอาจอยู่ในลำดับใดก็ได้ และยังไม่สะดวกต่อการค้นหาหรือการนำเสนอ จุดเริ่มต้นของการทำความเข้าใจคือมองข้อมูลเป็นรายการที่มีสมาชิกหลายตัว เช่น คะแนน ชื่อ หรือรหัสนักศึกษา จากนั้นกำหนดกติกาว่าจะใช้ค่าใดเป็นเกณฑ์ในการจัดลำดับ

ตัวอย่างการเรียนรู้

รายการคะแนน [72, 55, 91, 68] ยังไม่เรียง เมื่อกำหนดให้เรียงจากน้อยไปมาก ผลลัพธ์ที่ต้องการคือ [55, 68, 72, 91]

วิธีคิดทีละลำดับ

- อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย
- ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา
- ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง
- ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
ข้อมูล = [72, 55, 91, 68]
# ข้อมูลยังไม่เรียง
ผลลัพธ์ที่ต้องการ = [55, 68, 72, 91]
```

ประเด็นที่ควรตรวจสอบ

- แยกให้ออกระหว่างข้อมูลนำเข้าและผลลัพธ์

ระบุเกณฑ์ที่ใช้เรียงทุกครั้ง

ตรวจสอบว่าข้อมูลเดิมมีสมาชิกครบหลังการเรียง

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งชั้น

1.1.1.2 ความสัมพันธ์ระหว่างข้อมูลนำเข้าและผลลัพธ์

อัลกอริทึมการเรียงลำดับต้องรักษาสมาชิกของข้อมูลเดิมไว้

เพียงเปลี่ยนตำแหน่งของสมาชิกให้เป็นไปตามเงื่อนไข ดังนั้นข้อมูลนำเข้าและผลลัพธ์ควรมีจำนวนสมาชิกเท่ากัน และค่าทุกค่าควรปรากฏครบถ้วน เว้นแต่โจทย์จะระบุให้ตัดข้อมูลซ้ำ

ตัวอย่างการเรียนรู้

ถ้าข้อมูลนำเข้าคือ [4, 1, 4, 2] ผลลัพธ์แบบเรียงจากน้อยไปมากคือ [1, 2, 4, 4] เลข 4 ต้องยังคงอยู่สองครั้ง

วิธีคิดทีละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def ตรวจสอบความครบถ้วน(ก่อน, หลัง):  
    return len(ก่อน) == len(หลัง) and sorted(ก่อน) == sorted(หลัง)
```

ประเด็นที่ควรตรวจสอบ

จำนวนสมาชิกก่อนและหลังควรเท่ากัน

ข้อมูลซ้ำต้องไม่หายไปโดยไม่ตั้งใจ

การเรียงลำดับไม่ใช่การหาผลรวมของข้อมูล

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งชั้น

1.1.2 จุดประสงค์ของการเรียงลำดับข้อมูล

การเรียงลำดับช่วยให้ค้นหา นำเสนอ ตรวจสอบ และวิเคราะห์ข้อมูลได้เป็นระบบมากขึ้น

1.1.2.1 การเรียงลำดับช่วยให้ค้นหาข้อมูลได้ง่ายขึ้น

เมื่อข้อมูลเรียงแล้ว ผู้ใช้และอัลกอริทึมค้นหาสามารถคาดเดาตำแหน่งของข้อมูลได้ดีขึ้น
การค้นหาแบบทวิภาคเป็นตัวอย่างที่ต้องอาศัยข้อมูลเรียงลำดับ แต่ในระดับนี้ให้มองหลักง่าย ๆ
ว่ารายการที่เป็นระเบียบทำให้การไล่ตรวจและการสังเกตค่าผิดปกติทำได้สะดวก

ตัวอย่างการเรียนรู้

รายชื่อนักศึกษาที่เรียงตามรหัสทำให้ตรวจว่ารหัสใดหายไปได้ง่ายกว่ารายชื่อที่วางแบบสุ่ม

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย
ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา
ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง
ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
รหัส = [1003, 1001, 1005, 1002]  
รหัสเรียงแล้ว = [1001, 1002, 1003, 1005]
```

ประเด็นที่ควรตรวจสอบ

ระบุว่าจัดเรียงเพื่อค้นหาอะไร
พิจารณาค่าใช้จ่ายของการเรียงก่อนค้นหา
การเรียงไม่ได้ทำให้ข้อมูลเพิ่มขึ้น
ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

1.1.2.2 การเรียงลำดับช่วยนำเสนอและวิเคราะห์ข้อมูล

ตารางที่เรียงตามคะแนน ราคา หรือเวลา ช่วยให้เห็นแนวโน้มและอันดับได้เร็วขึ้น
ในงานจริงการเรียงอาจเป็นขั้นตอนเตรียมข้อมูลก่อนทำรายงาน
ไม่ได้มีเป้าหมายเพียงเพื่อให้โปรแกรมทำงานเร็วขึ้น

ตัวอย่างการเรียนรู้

หากเรียงยอดขายรายเดือนจากมากไปน้อย ผู้จัดการจะเห็นเดือนที่มียอดสูงสุดในช่วงต้นของรายการทันที

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย
ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา
ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
ยอดขาย = [1200, 850, 1650, 940]
# เรียงจากมากไปน้อยเพื่อทำอันดับ
ยอดขาย.sort(reverse=True)
```

ประเด็นที่ควรตรวจสอบ

กำหนดทิศทางให้ตรงกับวัตถุประสงค์

ระวังการเรียงวันที่เป็นข้อความ

ตรวจสอบรูปแบบตัวเลขก่อนนำไปเรียง

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งชั้น

1.2 รูปแบบและชนิดของข้อมูล

การเลือกทิศทางและชนิดข้อมูลมีผลโดยตรงต่อเงื่อนไขการเปรียบเทียบและผลลัพธ์

1.2.1 ทิศทางการเรียงลำดับ

ผู้เขียนอัลกอริทึมต้องกำหนดให้ชัดว่าค่าจะเพิ่มขึ้นหรือลดลงเมื่ออ่านจากซ้ายไปขวา

1.2.1.1 การเรียงจากน้อยไปมาก

การเรียงจากน้อยไปมากเรียกได้ว่า ascending order ค่าที่เล็กกว่าจะอยู่ทางซ้ายหรืออยู่ก่อนค่าที่ใหญ่กว่า การเปรียบเทียบหลักคือถ้าสมาชิกด้านซ้ายมากกว่าสมาชิกด้านขวา แสดงว่ายังต้องแก้ลำดับ

ตัวอย่างการเรียนรู้

[7, 2, 5, 1] เรียงจากน้อยไปมากเป็น [1, 2, 5, 7]

วิธีคิดทีละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def น้อยไปมาก(a):  
    return sorted(a)
```

ประเด็นที่ควรตรวจสอบ

ใช้เครื่องหมาย > เมื่อตรวจคู่ที่ผิดลำดับ

ศูนย์เป็นค่าที่ต้องนำมาพิจารณาตามปกติ

อย่าสับสนกับตำแหน่ง index

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

1.2.1.2 การเรียงจากมากไปน้อย

การเรียงจากมากไปน้อยเรียกได้ว่า descending order ค่าที่ใหญ่กว่าจะอยู่ก่อนค่าที่เล็กกว่า

วิธีสร้างอัลกอริทึมทำได้โดยเปลี่ยนเงื่อนไขการเปรียบเทียบ หรือเรียงจากน้อยไปมากแล้วกลับลำดับผลลัพธ์

ตัวอย่างการเรียนรู้

[7, 2, 5, 1] เรียงจากมากไปน้อยเป็น [7, 5, 2, 1]

วิธีคิดทีละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง
ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def มากไปน้อย(a):  
    return sorted(a, reverse=True)
```

ประเด็นที่ควรตรวจสอบ

ใช้ reverse ให้ตรงกับความหมายของโจทย์

ทดสอบด้วยข้อมูลที่มีค่าซ้ำ

ตรวจสอบว่าค่ามากที่สุดอยู่ตำแหน่งแรก

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

1.2.2 ชนิดของข้อมูลที่นำมาเรียง

ตัวเลขและข้อความมีกติกากาการเปรียบเทียบต่างกัน จึงต้องตรวจสอบชนิดข้อมูลก่อนเริ่มทำงาน

1.2.2.1 การเรียงข้อมูลตัวเลข

ตัวเลขสามารถเปรียบเทียบตามค่าทางคณิตศาสตร์โดยตรง แต่ต้องแยกจำนวนเต็มและจำนวนจริงให้เหมาะสม และต้องไม่เปรียบเทียบตัวเลขในรูปข้อความโดยไม่ตรวจสอบชนิดข้อมูล เพราะข้อความ '10' อาจถูกจัดก่อน '2' ตามลำดับตัวอักษร

ตัวอย่างการเรียนรู้

ตัวเลข [2, 10, 3] ต้องได้ [2, 3, 10] แต่ข้อความ ['2', '10', '3'] อาจได้ลำดับต่างกันหากไม่แปลงชนิด

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
ตัวเลข = [2, 10, 3]
print(sorted(ตัวเลข)) # [2, 3, 10]
```

ประเด็นที่ควรตรวจสอบ

ตรวจสอบชนิดข้อมูลก่อนเรียง

พิจารณาค่าติดลบและศูนย์

อย่าใช้การเทียบตัวอักษรกับตัวเลขโดยไม่ตั้งใจ

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

1.2.2.2 การเรียงข้อมูลตัวอักษรและข้อความ

การเรียงข้อความอาศัยลำดับรหัสตัวอักษรหรือกติกากาภาษาที่ระบบกำหนด

ในระดับเบื้องต้นควรใช้ข้อความภาษาอังกฤษเพื่อเห็นหลักชัดเจน

และควรอธิบายว่าตัวพิมพ์ใหญ่กับตัวพิมพ์เล็กอาจมีลำดับต่างกัน

ส่วนภาษาไทยต้องเลือกวิธีเปรียบเทียบที่เหมาะสมกับระบบ

ตัวอย่างการเรียนรู้

['Mango', 'Apple', 'Banana'] เรียงตามตัวอักษรได้ ['Apple', 'Banana', 'Mango']

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง
ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
ผลไม้ = ['Mango', 'Apple', 'Banana']  
print(sorted(ผลไม้))
```

ประเด็นที่ควรตรวจสอบ

กำหนดว่าจะสนใจตัวพิมพ์เล็กหรือใหญ่

ระวังช่องว่างที่ซ่อนอยู่ในข้อความ

ระบุเกณฑ์เมื่อข้อความมีหลายคำ

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

1.3 การดำเนินการพื้นฐาน

อัลกอริทึมการเรียงลำดับประกอบด้วย การเปรียบเทียบ และการเปลี่ยนตำแหน่งของสมาชิก

1.3.1 การเปรียบเทียบข้อมูล

เงื่อนไขมากกว่า น้อยกว่า และเท่ากัน เป็นพื้นฐานในการตัดสินใจว่าสมาชิกคู่หนึ่งอยู่ถูกลำดับหรือไม่

1.3.1.1 การเปรียบเทียบด้วยเครื่องหมายมากกว่าและน้อยกว่า

การเรียงลำดับเกิดจากการเปรียบเทียบสมาชิกเป็นคู่ เครื่องหมาย $>$ ใช้ตรวจสอบว่าค่าซ้ายมากกว่าค่าขวาหรือไม่ ส่วน $<$ ใช้ตรวจสอบว่าค่าซ้ายน้อยกว่าค่าขวาหรือไม่ การเลือกเครื่องหมายผิดจะทำให้ทิศทางการเรียงกลับด้าน

ตัวอย่างการเรียนรู้

ถ้าพบคู่ $[8, 3]$ ในการเรียงจากน้อยไปมาก เพราะ $8 > 3$ จึงต้องสลับตำแหน่ง

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
ซ้าย, ขวา = 8, 3
if ซ้าย > ขวา:
    ซ้าย, ขวา = ขวา, ซ้าย
```

ประเด็นที่ควรตรวจสอบ

ทดสอบด้วยคู่ที่เรียงถูกและผิด

เขียนเหตุผลของเงื่อนไขก่อนเขียนโค้ด

อย่าตีความ $>$ เป็นการกำหนดค่า

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ $[7, 3, 5, 1]$ หนึ่งขั้น

1.3.1.2 การตรวจสอบข้อมูลที่มีค่าเท่ากัน

เมื่อสมาชิกสองตัวเท่ากัน จะไม่มีเหตุผลให้สลับในอัลกอริทึมทั่วไป

การไม่สลับค่าที่เท่ากันช่วยลดการทำงานและช่วยรักษาลำดับเดิมของข้อมูลที่มีคีย์เท่ากัน

ซึ่งเกี่ยวข้องกับคุณสมบัติ stable sort

ตัวอย่างการเรียนรู้

ข้อมูล $[4A, 4B, 2]$ หากคีย์คือเลข 4 ควรได้ $[2, 4A, 4B]$ เมื่อรักษาลำดับเดิมของ 4A ก่อน 4B

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา
ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง
ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
if ชาย > ขว: # ไม่ใช่ >=  
    swap()
```

ประเด็นที่ควรตรวจสอบ

ใช่ > เมื่อไม่ต้องการสลับค่าที่เท่ากัน

แยกค่าที่ย่อออกจากข้อมูลประกอบ

ตรวจสอบลำดับของสมาชิกที่คีย์เท่ากัน

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

1.3.2 การเปลี่ยนตำแหน่งข้อมูล

เมื่อพบข้อมูลผิดลำดับ อัลกอริทึมอาจสลับสมาชิกหรือเลื่อนสมาชิกเพื่อสร้างตำแหน่งที่ถูกต้อง

1.3.2.1 การสลับตำแหน่งข้อมูล

การสลับคือการนำค่าจากสองตำแหน่งมาแลกกัน มักต้องใช้ตัวแปรชั่วคราวเพื่อไม่ให้ค่าหนึ่งถูกเขียนทับ การสลับเป็นแกนสำคัญของ Bubble และ Selection Sort

ตัวอย่างการเรียนรู้

สลับ [5, 2] ให้ได้ [2, 5] โดยเก็บ 5 ไว้ชั่วคราวก่อนนำ 2 มาวางด้านหน้า

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
ชั่วคราว = a[i]
a[i] = a[j]
a[j] = ชั่วคราว
```

ประเด็นที่ควรตรวจสอบ

ต้องเก็บค่าก่อนเขียนทับ

ตรวจสอบ index ให้ไม่เกินขนาดรายการ

ทดสอบกรณี i และ j เป็นตำแหน่งเดียวกัน

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

1.3.2.2 การเลื่อนตำแหน่งข้อมูล

การเลื่อนต่างจากการสลับตรงที่สมาชิกหลายตัวขยับไปในทิศทางเดียวกันเพื่อเปิดช่องว่าง การเลื่อนเป็นหัวใจของ Insertion Sort ซึ่งมักลดจำนวนการสลับได้เมื่อเทียบกับการแลกค่าทีละคู่

ตัวอย่างการเรียนรู้

ใน [2, 5, 7, 3] เมื่อนำ 3 ไปแทรก ต้องเลื่อน 7 และ 5 ไปทางขวาก่อนวาง 3 หลัง 2

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
key = 3
# เลื่อนค่าที่มากกว่า key ไปทางขวา
# แล้ววาง key ในช่องว่าง
```

ประเด็นที่ควรตรวจสอบ

เลื่อนจากด้านขวาไปซ้ายเพื่อป้องกันค่าหาย

เก็บ key ไว้ก่อนเริ่มเลื่อน

หยุดเมื่อพบค่าที่ไม่มากกว่า key

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งชั้น

1.4 การประเมินอัลกอริทึม

นอกจากผลลัพธ์ถูกต้องแล้ว ควรประเมินเวลาทำงาน หน่วยความจำ และคุณสมบัติการรักษาลำดับเดิม

1.4.1 ประสิทธิภาพด้านเวลา

การวิเคราะห์กรณีที่ดีที่สุด ค่าเฉลี่ย และกรณีแย่ที่สุดช่วยให้คาดการณ์พฤติกรรมเมื่อข้อมูลมีขนาดใหญ่มากขึ้น

1.4.1.1 กรณีที่ดีที่สุด ค่าเฉลี่ย และกรณีแย่ที่สุด

ประสิทธิภาพของอัลกอริทึมขึ้นกับรูปแบบข้อมูลด้วย กรณีที่ดีที่สุดคือข้อมูลเอื้อต่ออัลกอริทึม

กรณีแย่ที่สุดคือข้อมูลทำให้ต้องทำงานมาก

ส่วนกรณีเฉลี่ยใช้บอกพฤติกรรมโดยประมาณเมื่อข้อมูลมีหลายรูปแบบ

ตัวอย่างการเรียนรู้

Insertion Sort ทำงานดีเมื่อข้อมูลเกือบเรียงแล้ว แต่ทำงานมากเมื่อข้อมูลเรียงกลับทิศทาง

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

กรณีที่ดีที่สุด: ข้อมูลเกือบถูกต้อง
กรณีแย่ที่สุด: ข้อมูลเรียงย้อนกลับ

ประเด็นที่ควรตรวจสอบ

แยก best, average และ worst ให้ชัด

อย่าสรุปจากตัวอย่างชุดเดียว

ระบุว่ากำลังนับการเปรียบเทียบหรือการสลับ

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

1.4.1.2 ความหมายเบื้องต้นของ Big O

Big O ใช้อธิบายอัตราการเติบโตของเวลาเมื่อจำนวนข้อมูล n เพิ่มขึ้น ในระดับปี 1

ให้มองเป็นเครื่องมือเปรียบเทียบ เช่น $O(n)$ โตเป็นเส้นตรง ส่วน $O(n^2)$ โตเร็วกว่าเมื่อข้อมูลมีจำนวนมาก

ไม่จำเป็นต้องคำนวณเวลาจริงเป็นวินาที

ตัวอย่างการเรียนรู้

ถ้าอัลกอริทึมมีลูปลูปซ้อนสองชั้นที่วนตาม n มักพิจารณาเป็น $O(n^2)$

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
for i in range(n):  
    for j in range(n):  
        เปรียบเทียบ() # โดยย่อเป็น  $O(n^2)$ 
```

ประเด็นที่ควรตรวจสอบ

พิจารณาจำนวนรอบเมื่อ n โต

ตัดค่าคงที่ออกเมื่อเขียน Big O

อย่าสับสน $O(n^2)$ กับ n ยกกำลังสองของข้อมูล

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

1.4.2 คุณสมบัติของอัลกอริทึม

หน่วยความจำเพิ่มเติมและความเป็น Stable Sort เป็นเกณฑ์สำคัญในการเลือกวิธีเรียงข้อมูล

1.4.2.1 หน่วยความจำและ In-place Sort

บางอัลกอริทึมใช้พื้นที่เพิ่มเติมเพียงเล็กน้อยและแก้ไขข้อมูลในรายการเดิม เรียกว่า in-place ในทางกลับกัน Merge Sort ต้องใช้รายการช่วยสำหรับการผสานในรูปแบบพื้นฐาน จึงใช้หน่วยความจำมากกว่าอัลกอริทึมสามแบบแรก

ตัวอย่างการเรียนรู้

Bubble, Selection และ Insertion มักทำงานในรายการเดิม ส่วน Merge ต้องเตรียมพื้นที่เก็บผลการผสาน

วิธีคิดที่ละเอียด

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย
ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา
ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง
ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
# แนวคิด in-place  
a[i], a[j] = a[j], a[i]
```

ประเด็นที่ควรตรวจสอบ

แยกเวลาทำงานออกจากพื้นที่หน่วยความจำ
ระบุว่าพื้นที่เพิ่มเติมคิดรวมข้อมูลนำเข้าหรือไม่
การใช้พื้นที่เพิ่มอาจแลกกับความเร็วที่ดีขึ้น

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

1.4.2.2 ความหมายของ Stable Sort

Stable Sort รักษาลำดับเดิมของข้อมูลที่มีคีย์เท่ากัน คุณสมบัตินี้สำคัญเมื่อข้อมูลเป็นระเบียบ เช่น นักศึกษาหลายคนมีคะแนนเท่ากันแต่ต้องคงลำดับตามชื่อเดิมไว้ การตรวจสอบ stability ต้องใช้ข้อมูลที่มีทั้งคีย์และป้ายกำกับ

ตัวอย่างการเรียนรู้

[(80,'ก'), (70,'ข'), (80,'ค')] เมื่อเรียงตามคะแนนจากน้อยไปมาก ลำดับของ ก และ ค ควรยังเป็น ก ก่อน ค

วิธีคิดที่ละเอียด

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย
ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง
ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
ข้อมูล = [(80, 'ก'), (70, 'ข'), (80, 'ค')]  
# เรียงด้วย key=คะแนน และตรวจสอบลำดับป้ายกำกับ
```

ประเด็นที่ควรตรวจสอบ

- กำหนดคีย์ที่ใช้เปรียบเทียบ
- ทดสอบด้วยค่าซ้ำ
- อย่าตัดสิน stability จากผลลัพธ์ที่ไม่มีป้ายกำกับ
- ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

2 Bubble Sort

บทนี้อธิบายการเปรียบเทียบสมาชิกที่อยู่ติดกัน การสลับตำแหน่ง การทำงานเป็นรอบ และการปรับปรุงด้วยตรงตรวจสอบการสลับ

ผลลัพธ์การเรียนรู้ของบท

- อธิบายแนวคิดด้วยภาษาของตนเองได้
- ติดตามขั้นตอนจากตัวอย่างข้อมูลขนาดเล็กได้
- อ่าน Pseudocode และโค้ดภาษา Python ได้
- ตรวจสอบผลลัพธ์และอธิบายข้อผิดพลาดได้

2.1 แนวคิดพื้นฐานของ Bubble Sort

ส่วนนี้อธิบายกลไกหลักของ Bubble Sort และสถานะของข้อมูลระหว่างที่อัลกอริทึมทำงาน

2.1.1 กลไกการทำงาน

ผู้เรียนจะพิจารณาว่าอัลกอริทึมเลือก เปรียบเทียบ สลับ หรือเลื่อนสมาชิกอย่างไร

2.1.1.1 ความหมายและแนวคิดหลักของ Bubble Sort

Bubble Sort เปรียบเทียบสมาชิกที่อยู่ติดกันและสลับเมื่ออยู่ผิดลำดับ หลังผ่านข้อมูลหนึ่งรอบ ค่าที่มากที่สุดของส่วนที่ยังไม่เรียงจะค่อย ๆ เคลื่อนไปด้านท้าย จึงเหมือนฟองอากาศที่ลอยขึ้น

ตัวอย่างการเรียนรู้

ข้อมูล [5, 2, 4, 1] เมื่อเปรียบเทียบคู่ติดกันจากซ้ายไปขวา รอบแรกจะค่อย ๆ ดันค่า 5 ไปอยู่ท้ายรายการ

วิธีคิดที่ละลำดับ

- อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย
- ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา
- ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง
- ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
for pass in range(n - 1):
    for i in range(n - 1 - pass):
        if a[i] > a[i + 1]:
            a[i], a[i + 1] = a[i + 1], a[i]
```

ประเด็นที่ควรตรวจสอบ

- อธิบายขั้นตอนโดยไม่พึ่งโค้ดได้
- ระบุส่วนของข้อมูลที่เรียงแล้วและยังไม่เรียง

ใช้คำว่าเปรียบเทียบ สลับ หรือเลื่อนให้ถูกบริบท

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งชั้น

2.1.1.2 เจาะไขการเรียงจากน้อยไปมากด้วย Bubble Sort

การเรียงจากน้อยไปมากของ Bubble Sort ต้องทำให้ค่าที่เล็กกว่าอยู่ก่อนค่าที่ใหญ่กว่า โดยเลือกเงื่อนไขการเปรียบเทียบให้ตรงกับกลไกของอัลกอริทึม ในแต่ละรอบ

ตัวอย่างการเรียนรู้

ข้อมูล [5, 2, 4, 1] เมื่อเปรียบเทียบคู่ติดกันจากซ้ายไปขวา รอบแรกจะค่อย ๆ ดันค่า 5 ไปอยู่ท้ายรายการ

วิธีคิดทีละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละชั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def bubble_sort(a):  
    a = a.copy()  
    for end in range(len(a)-1, 0, -1):  
        for i in range(end):  
            if a[i] > a[i+1]:  
                a[i], a[i+1] = a[i+1], a[i]  
    return a
```

ประเด็นที่ควรตรวจสอบ

ตรวจค่าต่ำสุดและค่าสูงสุดหลังทำงาน

ทดสอบข้อมูลที่มีค่าซ้ำ

ตรวจว่าทิศทางผลลัพธ์ตรงกับโจทย์

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งชั้น

2.1.2 สถานะข้อมูลในแต่ละรอบ

การบันทึกสถานะก่อนและหลังแต่ละรอบทำให้เห็นว่าส่วนใดเรียงเสร็จแล้วและส่วนใดยังต้องตรวจต่อ

2.1.2.1 การทำงานทีละรอบของ Bubble Sort

การติดตามทีละรอบช่วยให้ผู้เรียนเห็นสถานะของรายการก่อนและหลังการดำเนินการหนึ่งครั้ง
ควรบันทึกค่าที่เปลี่ยน ตำแหน่งที่เปลี่ยน และส่วนที่ถือว่าเรียงเสร็จแล้ว ในแต่ละรอบ

ตัวอย่างการเรียนรู้

ข้อมูล [5, 2, 4, 1] เมื่อเปรียบเทียบคู่ติดกันจากซ้ายไปขวา รอบแรกจะค่อย ๆ ดันค่า 5 ไปอยู่ท้ายรายการ

วิธีคิดทีละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
for pass in range(n - 1):  
    for i in range(n - 1 - pass):  
        if a[i] > a[i + 1]:  
            a[i], a[i + 1] = a[i + 1], a[i]
```

ประเด็นที่ควรตรวจสอบ

เขียนรายการหลังจบรอบทุกครั้ง

ระบุจำนวนรอบสูงสุด

อย่าข้ามการตรวจข้อมูลที่ยังไม่เรียง

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

2.1.2.2 การทำงานกับข้อมูลที่เรียงแล้วหรือมีค่าซ้ำ

ข้อมูลที่เรียงแล้วและข้อมูลที่มีค่าซ้ำเป็นกรณีทดสอบสำคัญ

เพราะช่วยตรวจว่าอัลกอริทึมหยุดได้ถูกต้องและไม่ทำลายลำดับที่ไม่จำเป็น ในแต่ละรอบ

ตัวอย่างการเรียนรู้

ข้อมูล [1, 2, 2, 3] ควรได้ผลลัพธ์เดิมและไม่เกิดการสลับที่เกินจำเป็น

วิธีคิดทีละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def bubble_sort(a):  
    a = a.copy()  
    for end in range(len(a)-1, 0, -1):  
        for i in range(end):  
            if a[i] > a[i+1]:  
                a[i], a[i+1] = a[i+1], a[i]  
    return a
```

ประเด็นที่ควรตรวจสอบ

ทดสอบรายการว่างและรายการมีสมาชิกเดียว

ทดสอบค่าซ้ำทั้งหมด

พิจารณาการหยุดก่อนครบทุกชั้นเมื่อทำได้

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งชั้น

2.2 ขั้นตอนการทำงานและการติดตามผลลัพธ์

ส่วนนี้แสดงการทำงานกับข้อมูลจริงตั้งแต่รอบแรกจนถึงผลลัพธ์สุดท้าย รวมถึงข้อมูลกรณีพิเศษ

2.2.1 การเรียงจากน้อยไปมาก

เริ่มจากทิศทางที่ใช้บ่อยที่สุด แล้วติดตามการเปลี่ยนแปลงของรายการในแต่ละรอบ

2.2.1.1 ตัวอย่างติดตามผลลัพธ์ของ Bubble Sort รอบที่ 1

การทำตารางติดตามช่วยเชื่อมโยงแนวคิดกับผลลัพธ์จริง

ให้เขียนข้อมูลทุกครั้งที่มีการเปลี่ยนแปลงและทำเครื่องหมายส่วนที่เรียงเสร็จแล้ว

ตัวอย่างการเรียนรู้

ข้อมูล [5, 2, 4, 1] เมื่อเปรียบเทียบคู่ติดกันจากซ้ายไปขวา รอบแรกจะค่อย ๆ ดันค่า 5 ไปอยู่ท้ายรายการ

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
for pass in range(n - 1):
    for i in range(n - 1 - pass):
        if a[i] > a[i + 1]:
            a[i], a[i + 1] = a[i + 1], a[i]
```

ประเด็นที่ควรตรวจสอบ

บันทึกค่าก่อนเริ่มรอบ

บันทึกค่าหลังจบการดำเนินการ

ตรวจสอบว่าส่วนที่ประกาศว่าเรียงแล้วถูกต้อง

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

2.2.1.2 ตัวอย่างติดตามผลลัพธ์ของ Bubble Sort รอบสุดท้าย

รอบสุดท้ายควรทำให้เห็นว่ารายการครบถ้วนและไม่มีคู่ข้อมูลที่ผิดเงื่อนไข

การอธิบายรอบสุดท้ายช่วยให้สร้างคำถามตรวจสอบความเข้าใจได้

ตัวอย่างการเรียนรู้

ข้อมูล [5, 2, 4, 1] เมื่อเปรียบเทียบคู่ติดกันจากซ้ายไปขวา รอบแรกจะค่อย ๆ ดันค่า 5 ไปอยู่ท้ายรายการ

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา
ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง
ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def bubble_sort(a):  
    a = a.copy()  
    for end in range(len(a)-1, 0, -1):  
        for i in range(end):  
            if a[i] > a[i+1]:  
                a[i], a[i+1] = a[i+1], a[i]  
    return a
```

ประเด็นที่ควรตรวจสอบ

ตรวจสอบผลลัพธ์ด้วยการใส่คู่ข้อมูล

ตรวจสอบจำนวนสมาชิก

ตรวจสอบข้อมูลที่ซ้ำและค่าขอบเขต

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

2.2.2 รูปแบบข้อมูลพิเศษ

การเรียงจากมากไปน้อย ข้อมูลว่าง ข้อมูลซ้ำ และสมาชิกเดียวใช้ตรวจสอบความสมบูรณ์ของอัลกอริทึม

2.2.2.1 การดัดแปลงให้เรียงจากมากไปน้อยด้วย Bubble Sort

การเปลี่ยนทิศทางมักทำได้โดยกลับเงื่อนไขการเปรียบเทียบ หรือกลับผลลัพธ์หลังเรียง

วิธีที่เหมาะสมขึ้นกับโครงสร้างอัลกอริทึมและเป้าหมายด้าน stability

ตัวอย่างการเรียนรู้

ข้อมูล [3, 1, 4, 2] ต้องได้ [4, 3, 2, 1] เมื่อกำหนด descending

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def bubble_sort(a):  
    a = a.copy()  
    for end in range(len(a)-1, 0, -1):  
        for i in range(end):  
            if a[i] > a[i+1]:  
                a[i], a[i+1] = a[i+1], a[i]  
    return a
```

ประเด็นที่ควรตรวจสอบ

เปลี่ยนเงื่อนไขอย่างเป็นระบบ

ทดสอบข้อมูลที่ซ้ำ

ระวังการกลับรายการหลังมีการแก้ข้อมูลต้นฉบับ

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

2.2.2.2 กรณีข้อมูลว่าง สมาชิกเดียว และสมาชิกซ้ำของ Bubble Sort

กรณีขอบเขตช่วยป้องกันข้อผิดพลาดจาก loop และ index รายการว่างควรคืนผลลัพธ์ว่าง

รายการที่มีสมาชิกเดียวไม่ต้องดำเนินการ และรายการที่มีสมาชิกซ้ำต้องยังคงจำนวนสมาชิกครบ

ตัวอย่างการเรียนรู้

[], [9] และ [2, 2, 1, 1] เป็นชุดข้อมูลทดสอบขั้นต่ำที่ควรมี

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def bubble_sort(a):  
    a = a.copy()  
    for end in range(len(a)-1, 0, -1):  
        for i in range(end):  
            if a[i] > a[i+1]:  
                a[i], a[i+1] = a[i+1], a[i]  
    return a
```

ประเด็นที่ควรตรวจสอบ

ไม่เข้าถึง index ของรายการว่าง

ไม่วนเกินขอบเขต

ตรวจสอบว่าค่าซ้ำไม่หาย

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

2.3 การเขียนอัลกอริทึมและโปรแกรม

จากแนวคิดจะถูกแปลงเป็น Pseudocode และ Python เพื่อให้ทดลองและตรวจสอบข้อผิดพลาดได้

2.3.1 Pseudocode และโค้ดภาษา Python

Pseudocode ช่วยตรวจสอบตรรกะ ส่วน Python ช่วยทดลองอัลกอริทึมกับข้อมูลขนาดเล็ก

2.3.1.1 Pseudocode ของ Bubble Sort

Pseudocode แสดงลำดับการทำงานโดยไม่ผูกกับภาษาโปรแกรมใดภาษาหนึ่ง

มีประโยชน์ต่อการตรวจสอบตรรกะก่อนเขียนโปรแกรมจริง ในแต่ละรอบ

ตัวอย่างการเรียนรู้

เริ่มจากรับรายการ กำหนดรอบหรือขอบเขต เปรียบเทียบข้อมูล ดำเนินการตามเงื่อนไข และคืนค่ารายการที่เรียงแล้ว

วิธีคิดทีละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
for pass in range(n - 1):
    for i in range(n - 1 - pass):
        if a[i] > a[i + 1]:
            a[i], a[i + 1] = a[i + 1], a[i]
```

ประเด็นที่ควรตรวจสอบ

กำหนดตัวแปรให้สื่อความหมาย

ระบุเงื่อนไขหยุด

แยกขั้นตอนหลักกับรายละเอียดการสลับหรือรวม

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

2.3.1.2 โค้ด Python ของ Bubble Sort

โค้ดภาษา Python ช่วยทดลองอัลกอริทึมด้วยข้อมูลขนาดเล็กได้ง่าย

การเขียนฟังก์ชันแยกจากส่วนรับข้อมูลทำให้ทดสอบซ้ำและนำไปสร้างคำถามได้สะดวก ในแต่ละรอบ

ตัวอย่างการเรียนรู้

ทดลองเรียง [5, 1, 4, 2] แล้วตรวจสอบด้วย assert ว่าผลลัพธ์เป็น [1, 2, 4, 5]

วิธีคิดทีละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def bubble_sort(a):  
    a = a.copy()  
    for end in range(len(a)-1, 0, -1):  
        for i in range(end):  
            if a[i] > a[i+1]:  
                a[i], a[i+1] = a[i+1], a[i]  
    return a
```

ประเด็นที่ควรตรวจสอบ

ใช้ชื่อฟังก์ชันและตัวแปรชัดเจน

ทดสอบผลลัพธ์ด้วยข้อมูลหลายรูปแบบ

ไม่ใช่ sorted เป็นคำตอบแทนการฝึกเขียนอัลกอริทึม

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

2.3.2 การตรวจหาข้อผิดพลาดและการทดสอบ

การอ่านโค้ดและการทดสอบข้อมูลหลายรูปแบบช่วยยืนยันว่าผลลัพธ์ถูกต้องจริง

2.3.2.1 การอ่านโค้ดและค้นหาข้อผิดพลาดใน Bubble Sort

ผู้เรียนควรอ่านโค้ดโดยตามค่าของตัวแปรหลัก ไม่เดาจากชื่ออัลกอริทึมเพียงอย่างเดียว
ข้อผิดพลาดที่พบบ่อยคือขอบเขตลูป เงื่อนไขกลับด้าน และการไม่เก็บค่าก่อนสลับหรือเลื่อน

ตัวอย่างการเรียนรู้

ถ้าใช้ range ผิดหนึ่งตำแหน่ง อาจไม่ได้เปรียบเทียบสมาชิกคู่สุดท้ายหรือเกิด index เกินขอบเขต

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def bubble_sort(a):  
    a = a.copy()  
    for end in range(len(a)-1, 0, -1):  
        for i in range(end):  
            if a[i] > a[i+1]:  
                a[i], a[i+1] = a[i+1], a[i]  
    return a
```

ประเด็นที่ควรตรวจสอบ

ตรวจสอบจุดเริ่มและจุดสิ้นสุดของลูป

ตรวจสอบตัวแปรที่เปลี่ยนทุกครั้ง

สร้างข้อมูลเล็กเพื่อจำลองทีละบรรทัด

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

2.3.2.2 การทดสอบความถูกต้องของ Bubble Sort

การทดสอบความถูกต้องควรตรวจสอบเรื่องพร้อมกัน คือผลลัพธ์เรียงถูกต้องทิศทาง
และสมาชิกเดิมไม่หายหรือเพิ่มเกินมา

วิธีตรวจง่ายคือเปรียบเทียบกับผลลัพธ์ที่คาดหวังหรือใช้สำเนาข้อมูลต้นฉบับ

ตัวอย่างการเรียนรู้

หลังเรียง [4, 2, 4, 1] ให้ตรวจว่าได้ [1, 2, 4, 4] และจำนวนเลข 4 ยังเป็นสองตัว

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา
ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง
ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
ผล = อัลกอริทึม(ข้อมูล)  
assert ผล == [1, 2, 4, 4]
```

ประเด็นที่ควรตรวจสอบ

ใช้ข้อมูลปกติและข้อมูลขอบเขต

ทดสอบทั้ง ascending และ descending

บันทึกผลการทดสอบเพื่อทำ Dataset

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

2.4 การวิเคราะห์และประยุกต์ใช้

ส่วนสุดท้ายเชื่อมโยงอัลกอริทึมกับประสิทธิภาพ ข้อจำกัด และสถานการณ์ใช้งาน

2.4.1 ประสิทธิภาพและความซับซ้อน

พิจารณาจำนวนการดำเนินการและสรุปแนวโน้มด้วย Big O ในระดับที่เหมาะสมกับปี 1

2.4.1.1 การวิเคราะห์เวลาและจำนวนการดำเนินการของ Bubble Sort

การวิเคราะห์ควรแยกการเปรียบเทียบ การสลับ การเลื่อน หรือการรวมตามลักษณะอัลกอริทึม แล้วสรุปเป็นอัตราการเติบโตที่เข้าใจง่าย โดยทั่วไป $O(n^2)$; ถ้าเพิ่มรองตรวจสอบการสลับกรณีข้อมูลเรียงแล้วอาจหยุดได้เร็วขึ้น

ตัวอย่างการเรียนรู้

ข้อมูล [5, 2, 4, 1] เมื่อเปรียบเทียบคู่ติดกันจากซ้ายไปขวา รอบแรกจะค่อย ๆ ดันค่า 5 ไปอยู่ท้ายรายการ

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

จำนวนการดำเนินการขึ้นกับ n
สรุปเป็น Big O หลังตัดค่าคงที่และพจน์ที่มีผลน้อยกว่า

ประเด็นที่ควรตรวจสอบ

ระบุ n ว่าหมายถึงอะไร

ไม่ใช่ผลจากเวลาบนเครื่องเดียวเป็นคำอธิบายทั้งหมด

เปรียบเทียบกับอัลกอริทึมในบทอื่น

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

2.4.1.2 ข้อดี ข้อจำกัด และสถานการณ์ใช้งานของ Bubble Sort

ไม่มีอัลกอริทึมใดดีที่สุดทุกสถานการณ์ ต้องพิจารณาขนาดข้อมูล รูปแบบข้อมูล หน่วยความจำ และความต้องการรักษาลำดับเดิม เหมาะกับการเรียนแนวคิดการเปรียบเทียบและสลับข้อมูล แต่ไม่เหมาะกับข้อมูลขนาดใหญ่มาก

ตัวอย่างการเรียนรู้

เลือกวิธีเรียงที่เหมาะสมกับรายการขนาดเล็กเพื่อความเข้าใจ หรือเลือกวิธีที่โตช้ากว่าเมื่อข้อมูลมีจำนวนมาก

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
ถ้า len(ข้อมูล) < 2:  
    return ข้อมูล  
# เลือกอัลกอริทึมตามลักษณะงาน
```

ประเด็นที่ควรตรวจสอบ

อธิบายเหตุผลของการเลือกใช้

ระบุข้อจำกัดอย่างน้อยหนึ่งข้อ

แยกความง่ายในการเรียนรู้ออกจากประสิทธิภาพ

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

2.4.2 การเลือกใช้และการประยุกต์

ไม่มีวิธีใดที่ดีที่สุดทุกกรณี จึงต้องเลือกให้สัมพันธ์กับขนาดและชนิดของข้อมูล

2.4.2.1 โจทย์ฝึกประยุกต์ใช้ Bubble Sort

โจทย์ประยุกต์ควรเปลี่ยนจากตัวเลขธรรมดาเป็นข้อมูลที่มีความหมาย เช่น คะแนน ราคา หรือเวลา แต่ยังใช้แกนหลักของอัลกอริทึมเดิม

ตัวอย่างการเรียนรู้

จัดอันดับคะแนนนักเรียนจากสูงไปต่ำ โดยเก็บรหัสนักเรียนไว้คู่กับคะแนนและไม่ทำให้ข้อมูลคู่แยกจากกัน

วิธีคิดที่ละเอียด

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def bubble_sort(a):  
    a = a.copy()  
    for end in range(len(a)-1, 0, -1):  
        for i in range(end):  
            if a[i] > a[i+1]:  
                a[i], a[i+1] = a[i+1], a[i]  
    return a
```

ประเด็นที่ควรตรวจสอบ

ระบุคีย์ที่ใช้เรียง

รักษาข้อมูลประกอบของสมาชิก

อธิบายเหตุผลของทิศทางการเรียง

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

2.4.2.2 สรุปทบทวนและคำถามตรวจสอบความเข้าใจเรื่อง Bubble Sort

การสรุปควรตอบได้ว่าอัลกอริทึมเลือกหรือเปลี่ยนข้อมูลอย่างไร ใช้เวลาประมาณเท่าไร ใช้พื้นที่เพิ่มหรือไม่ และเหมาะกับงานลักษณะใด จากสรุปนี้สามารถสร้างคำถามแบบอธิบาย คำนวณ ติดตามผล และแก้โค้ดได้

ตัวอย่างการเรียนรู้

คำถามตัวอย่าง: ถ้าให้ข้อมูล [6, 3, 5, 1] อัลกอริทึมนี้ทำอะไรในรอบแรก และผลลัพธ์หลังจบรอบเป็นอย่างไร

วิธีคิดที่ละเอียด

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง
ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
คำถาม = 'อธิบายรอบแรกของ ' + ชื่ออัลกอริทึม  
คำตอบควรมีขั้นตอนและผลลัพธ์
```

ประเด็นที่ควรตรวจสอบ

คำตอบต้องอธิบายเหตุผล ไม่ใช่ตอบเฉพาะผลลัพธ์

เชื่อมโยงกับ Big O และข้อจำกัด

อ้างอิงหัวข้อย่อยที่ใช้ตอบได้

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

3 Selection Sort

บทนี้อธิบายการเลือกค่าน้อยที่สุดหรือมากที่สุดจากส่วนที่ยังไม่เรียง แล้วนำไปวางในตำแหน่งที่ถูกต้อง

ผลลัพธ์การเรียนรู้ของบท

อธิบายแนวคิดด้วยภาษาของตนเองได้

ติดตามขั้นตอนจากตัวอย่างข้อมูลขนาดเล็กได้

อ่าน Pseudocode และโค้ดภาษา Python ได้

ตรวจสอบผลลัพธ์และอธิบายข้อผิดพลาดได้

3.1 แนวคิดพื้นฐานของ Selection Sort

ส่วนนี้อธิบายกลไกหลักของ Selection Sort และสถานะของข้อมูลระหว่างที่อัลกอริทึมทำงาน

3.1.1 กลไกการทำงาน

ผู้เรียนจะพิจารณาว่าอัลกอริทึมเลือก เปรียบเทียบ สลับ หรือเลื่อนสมาชิกอย่างไร

3.1.1.1 ความหมายและแนวคิดหลักของ Selection Sort

Selection Sort แบ่งรายการเป็นส่วนที่เรียงแล้วและส่วนที่ยังไม่เรียง

ในแต่ละรอบจะค้นหาค่าที่เหมาะสมที่สุดจากส่วนที่ยังไม่เรียง แล้วนำไปวางในตำแหน่งถัดไป

ตัวอย่างการเรียนรู้

ข้อมูล [5, 2, 4, 1] ในการเรียงจากน้อยไปมาก รอบแรกค้นหา 1 แล้วสลับมาไว้ตำแหน่งแรก จากนั้นพิจารณาเฉพาะส่วนที่เหลือ

วิธีคิดทีละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
for start in range(n - 1):
    min_i = start
    for i in range(start + 1, n):
        if a[i] < a[min_i]:
            min_i = i
    a[start], a[min_i] = a[min_i], a[start]
```

ประเด็นที่ควรตรวจสอบ

อธิบายขั้นตอนโดยไม่พึ่งโค้ดได้

ระบุส่วนของข้อมูลที่เรียงแล้วและยังไม่เรียง

ใช้คำว่าเปรียบเทียบ สลับ หรือเลื่อนให้ถูกบริบท

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งชั้น

3.1.1.2 เจาะไขการเรียงจากน้อยไปมากด้วย Selection Sort

การเรียงจากน้อยไปมากของ Selection Sort ต้องทำให้ค่าที่เล็กกว่าอยู่ก่อนค่าที่ใหญ่กว่า โดยเลือกเจาะไขการเปรียบเทียบให้ตรงกับกลไกของอัลกอริทึม ในแต่ละรอบ

ตัวอย่างการเรียนรู้

ข้อมูล [5, 2, 4, 1] ในการเรียงจากน้อยไปมาก รอบแรกค้นหา 1 แล้วสลับมาไว้ตำแหน่งแรก จากนั้นพิจารณาเฉพาะส่วนที่เหลือ

วิธีคิดทีละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละชั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def selection_sort(a):
    a = a.copy()
    for start in range(len(a)-1):
        min_i = start
        for i in range(start+1, len(a)):
            if a[i] < a[min_i]:
                min_i = i
        a[start], a[min_i] = a[min_i], a[start]
    return a
```

ประเด็นที่ควรตรวจสอบ

ตรวจค่าต่ำสุดและค่าสูงสุดหลังทำงาน

ทดสอบข้อมูลที่มีค่าซ้ำ

ตรวจว่าทิศทางผลลัพธ์ตรงกับโจทย์

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งชั้น

3.1.2 สถานะข้อมูลในแต่ละรอบ

การบันทึกสถานะก่อนและหลังแต่ละรอบทำให้เห็นว่าส่วนใดเรียงเสร็จแล้วและส่วนใดยังต้องตรวจต่อ

3.1.2.1 การทำงานทีละรอบของ Selection Sort

การติดตามทีละรอบช่วยให้ผู้เรียนเห็นสถานะของรายการก่อนและหลังการดำเนินการหนึ่งครั้ง
ควรบันทึกค่าที่เปลี่ยน ตำแหน่งที่เปลี่ยน และส่วนที่ถือว่าเรียงเสร็จแล้ว ในแต่ละรอบ

ตัวอย่างการเรียนรู้

ข้อมูล [5, 2, 4, 1] ในการเรียงจากน้อยไปมาก รอบแรกค้นหา 1 แล้วสลับมาไว้ตำแหน่งแรก
จากนั้นพิจารณาเฉพาะส่วนที่เหลือ

วิธีคิดทีละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
for start in range(n - 1):  
    min_i = start  
    for i in range(start + 1, n):  
        if a[i] < a[min_i]:  
            min_i = i  
    a[start], a[min_i] = a[min_i], a[start]
```

ประเด็นที่ควรตรวจสอบ

เขียนรายการหลังจบรอบทุกครั้ง

ระบุจำนวนรอบสูงสุด

อย่าข้ามการตรวจข้อมูลที่ยังไม่เรียง

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

3.1.2.2 การทำงานกับข้อมูลที่เรียงแล้วหรือมีค่าซ้ำ

ข้อมูลที่เรียงแล้วและข้อมูลที่มีค่าซ้ำเป็นกรณีทดสอบสำคัญ

เพราะช่วยตรวจว่าอัลกอริทึมหยุดได้ถูกต้องและไม่ทำลายลำดับที่ไม่จำเป็น ในแต่ละรอบ

ตัวอย่างการเรียนรู้

ข้อมูล [1, 2, 2, 3] ควรได้ผลลัพธ์เดิมและไม่เกิดการสลับที่เกินจำเป็น

วิธีคิดทีละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา
ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง
ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def selection_sort(a):  
    a = a.copy()  
    for start in range(len(a)-1):  
        min_i = start  
        for i in range(start+1, len(a)):  
            if a[i] < a[min_i]:  
                min_i = i  
        a[start], a[min_i] = a[min_i], a[start]  
    return a
```

ประเด็นที่ควรตรวจสอบ

ทดสอบรายการว่างและรายการมีสมาชิกเดียว

ทดสอบค่าซ้ำทั้งหมด

พิจารณาการหยุดก่อนครบทุกขั้นเมื่อทำได้

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

3.2 ขั้นตอนการทำงานและการติดตามผลลัพธ์

ส่วนนี้แสดงการทำงานกับข้อมูลจริงตั้งแต่รอบแรกจนถึงผลลัพธ์สุดท้าย รวมถึงข้อมูลกรณีพิเศษ

3.2.1 การเรียงจากน้อยไปมาก

เริ่มจากทิศทางที่ใช้บ่อยที่สุด แล้วติดตามการเปลี่ยนแปลงของรายการในแต่ละรอบ

3.2.1.1 ตัวอย่างติดตามผลลัพธ์ของ Selection Sort รอบที่ 1

การทำตารางติดตามช่วยเชื่อมโยงแนวคิดกับผลลัพธ์จริง

ให้เขียนข้อมูลทุกครั้งที่มีการเปลี่ยนแปลงและทำเครื่องหมายส่วนที่เรียงเสร็จแล้ว

ตัวอย่างการเรียนรู้

ข้อมูล [5, 2, 4, 1] ในการเรียงจากน้อยไปมาก รอบแรกค้นหา 1 แล้วสลับมาไว้ตำแหน่งแรก
จากนั้นพิจารณาเฉพาะส่วนที่เหลือ

วิธีคิดทีละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
for start in range(n - 1):
    min_i = start
    for i in range(start + 1, n):
        if a[i] < a[min_i]:
            min_i = i
    a[start], a[min_i] = a[min_i], a[start]
```

ประเด็นที่ควรตรวจสอบ

บันทึกค่าก่อนเริ่มรอบ

บันทึกค่าหลังจบการดำเนินการ

ตรวจสอบว่าส่วนที่ประกาศว่าเรียงแล้วถูกต้อง

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

3.2.1.2 ตัวอย่างติดตามผลลัพธ์ของ Selection Sort รอบสุดท้าย

รอบสุดท้ายควรทำให้เห็นว่ารายการครบถ้วนและไม่มีคู่ข้อมูลที่ผิดเงื่อนไข

การอธิบายรอบสุดท้ายช่วยให้สร้างคำถามตรวจสอบความเข้าใจได้

ตัวอย่างการเรียนรู้

ข้อมูล [5, 2, 4, 1] ในการเรียงจากน้อยไปมาก รอบแรกค้นหา 1 แล้วสลับมาไว้ตำแหน่งแรก
จากนั้นพิจารณาเฉพาะส่วนที่เหลือ

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def selection_sort(a):  
    a = a.copy()  
    for start in range(len(a)-1):  
        min_i = start  
        for i in range(start+1, len(a)):  
            if a[i] < a[min_i]:  
                min_i = i  
        a[start], a[min_i] = a[min_i], a[start]  
    return a
```

ประเด็นที่ควรตรวจสอบ

ตรวจสอบผลลัพธ์ด้วยการใส่คู่ข้อมูล

ตรวจสอบจำนวนสมาชิก

ตรวจสอบข้อมูลที่ซ้ำและค่าขอบเขต

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

3.2.2 รูปแบบข้อมูลพิเศษ

การเรียงจากมากไปน้อย ข้อมูลว่าง ข้อมูลซ้ำ และสมาชิกเดียวใช้ตรวจสอบความสมบูรณ์ของอัลกอริทึม

3.2.2.1 การดัดแปลงให้เรียงจากมากไปน้อยด้วย Selection Sort

การเปลี่ยนทิศทางการทำซ้ำได้โดยกลับเงื่อนไขการเปรียบเทียบ หรือกลับผลลัพธ์หลังเรียง

วิธีที่เหมาะสมขึ้นกับโครงสร้างอัลกอริทึมและเป้าหมายด้าน stability

ตัวอย่างการเรียนรู้

ข้อมูล [3, 1, 4, 2] ต้องได้ [4, 3, 2, 1] เมื่อกำหนด descending

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def selection_sort(a):
    a = a.copy()
    for start in range(len(a)-1):
        min_i = start
        for i in range(start+1, len(a)):
            if a[i] < a[min_i]:
                min_i = i
        a[start], a[min_i] = a[min_i], a[start]
    return a
```

ประเด็นที่ควรตรวจสอบ

เปลี่ยนเงื่อนไขอย่างเป็นระบบ

ทดสอบข้อมูลที่ซ้ำ

ระวังการกลับรายการหลังมีการแก้ข้อมูลต้นฉบับ

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

3.2.2.2 กรณีข้อมูลว่าง สมาชิกเดียว และสมาชิกซ้ำของ Selection Sort

กรณีขอบเขตช่วยป้องกันข้อผิดพลาดจาก loop และ index รายการว่างควรคืนผลลัพธ์ว่าง

รายการที่มีสมาชิกเดียวไม่ต้องดำเนินการ และรายการที่มีสมาชิกซ้ำต้องยังคงจำนวนสมาชิกครบ

ตัวอย่างการเรียนรู้

[], [9] และ [2, 2, 1, 1] เป็นชุดข้อมูลทดสอบขั้นต่ำที่ควรมี

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา
ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง
ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def selection_sort(a):  
    a = a.copy()  
    for start in range(len(a)-1):  
        min_i = start  
        for i in range(start+1, len(a)):  
            if a[i] < a[min_i]:  
                min_i = i  
        a[start], a[min_i] = a[min_i], a[start]  
    return a
```

ประเด็นที่ควรตรวจสอบ

ไม่เข้าถึง index ของรายการว่าง

ไม่วนเกินขอบเขต

ตรวจสอบว่าค่าซ้ำไม่หาย

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

3.3 การเขียนอัลกอริทึมและโปรแกรม

จากแนวคิดจะถูกแปลงเป็น Pseudocode และ Python เพื่อให้ทดลองและตรวจสอบข้อผิดพลาดได้

3.3.1 Pseudocode และโค้ดภาษา Python

Pseudocode ช่วยตรวจสอบตรรกะ ส่วน Python ช่วยทดลองอัลกอริทึมกับข้อมูลขนาดเล็ก

3.3.1.1 Pseudocode ของ Selection Sort

Pseudocode แสดงลำดับการทำงานโดยไม่ผูกกับภาษาโปรแกรมใดภาษาหนึ่ง

มีประโยชน์ต่อการตรวจสอบตรรกะก่อนเขียนโปรแกรมจริง ในแต่ละรอบ

ตัวอย่างการเรียนรู้

เริ่มจากรับรายการ กำหนดรอบหรือขอบเขต เปรียบเทียบข้อมูล ดำเนินการตามเงื่อนไข และคืนค่ารายการที่เรียงแล้ว

วิธีคิดทีละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
for start in range(n - 1):
    min_i = start
    for i in range(start + 1, n):
        if a[i] < a[min_i]:
            min_i = i
    a[start], a[min_i] = a[min_i], a[start]
```

ประเด็นที่ควรตรวจสอบ

กำหนดตัวแปรให้สื่อความหมาย

ระบุเงื่อนไขหยุด

แยกขั้นตอนหลักกับรายละเอียดการสลับหรือรวม

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

3.3.1.2 โค้ด Python ของ Selection Sort

โค้ดภาษา Python ช่วยทดลองอัลกอริทึมด้วยข้อมูลขนาดเล็กได้ง่าย

การเขียนฟังก์ชันแยกจากส่วนรับข้อมูลทำให้ทดสอบซ้ำและนำไปสร้างคำถามได้สะดวก ในแต่ละรอบ

ตัวอย่างการเรียนรู้

ทดลองเรียง [5, 1, 4, 2] แล้วตรวจสอบด้วย assert ว่าผลลัพธ์เป็น [1, 2, 4, 5]

วิธีคิดทีละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def selection_sort(a):  
    a = a.copy()  
    for start in range(len(a)-1):  
        min_i = start  
        for i in range(start+1, len(a)):  
            if a[i] < a[min_i]:  
                min_i = i  
        a[start], a[min_i] = a[min_i], a[start]  
    return a
```

ประเด็นที่ควรตรวจสอบ

ใช้ชื่อฟังก์ชันและตัวแปรชัดเจน

ทดสอบผลลัพธ์ด้วยข้อมูลหลายรูปแบบ

ไม่ใช่ sorted เป็นคำตอบแทนการฝึกเขียนอัลกอริทึม

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

3.3.2 การตรวจหาข้อผิดพลาดและการทดสอบ

การอ่านโค้ดและการทดสอบข้อมูลหลายรูปแบบช่วยยืนยันว่าผลลัพธ์ถูกต้องจริง

3.3.2.1 การอ่านโค้ดและค้นหาข้อผิดพลาดใน Selection Sort

ผู้เรียนควรอ่านโค้ดโดยตามค่าของตัวแปรหลัก ไม่เดาจากชื่ออัลกอริทึมเพียงอย่างเดียว
ข้อผิดพลาดที่พบบ่อยคือขอบเขตลูป เงื่อนไขกลับด้าน และการไม่เก็บค่าก่อนสลับหรือเลื่อน

ตัวอย่างการเรียนรู้

ถ้าใช้ range ผิดหนึ่งตำแหน่ง อาจไม่ได้เปรียบเทียบสมาชิกคู่สุดท้ายหรือเกิด index เกินขอบเขต

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def selection_sort(a):  
    a = a.copy()  
    for start in range(len(a)-1):  
        min_i = start  
        for i in range(start+1, len(a)):  
            if a[i] < a[min_i]:  
                min_i = i  
        a[start], a[min_i] = a[min_i], a[start]  
    return a
```

ประเด็นที่ควรตรวจสอบ

ตรวจสอบจุดเริ่มและจุดสิ้นสุดของลูป

ตรวจสอบตัวแปรที่เปลี่ยนทุกครั้ง

สร้างข้อมูลเล็กเพื่อจำลองทีละบรรทัด

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

3.3.2.2 การทดสอบความถูกต้องของ Selection Sort

การทดสอบความถูกต้องควรตรวจสอบเรื่องพร้อมกัน คือผลลัพธ์เรียงถูกต้องทิศทาง
และสมาชิกเดิมไม่หายหรือเพิ่มเกินมา

วิธีตรวจง่ายคือเปรียบเทียบกับผลลัพธ์ที่คาดหวังหรือใช้สำเนาข้อมูลต้นฉบับ

ตัวอย่างการเรียนรู้

หลังเรียง [4, 2, 4, 1] ให้ตรวจว่าได้ [1, 2, 4, 4] และจำนวนเลข 4 ยังเป็นสองตัว

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
ผล = อัลกอริทึม(ข้อมูล)
assert ผล == [1, 2, 4, 4]
```

ประเด็นที่ควรตรวจสอบ

ใช้ข้อมูลปกติและข้อมูลขอบเขต

ทดสอบทั้ง ascending และ descending

บันทึกผลการทดสอบเพื่อทำ Dataset

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

3.4 การวิเคราะห์และประยุกต์ใช้

ส่วนสุดท้ายเชื่อมโยงอัลกอริทึมกับประสิทธิภาพ ข้อจำกัด และสถานการณ์ใช้งาน

3.4.1 ประสิทธิภาพและความซับซ้อน

พิจารณาจำนวนการดำเนินการและสรุปแนวโน้มด้วย Big O ในระดับที่เหมาะสมกับปี 1

3.4.1.1 การวิเคราะห์เวลาและจำนวนการดำเนินการของ Selection Sort

การวิเคราะห์ควรแยกการเปรียบเทียบ การสลับ การเลื่อน หรือการรวมตามลักษณะอัลกอริทึม แล้วสรุปเป็นอัตราการเติบโตที่เข้าใจง่าย $O(n^2)$ ในการเปรียบเทียบเกือบทุกกรณี แต่จำนวนการสลับมักน้อยกว่า Bubble Sort

ตัวอย่างการเรียนรู้

ข้อมูล [5, 2, 4, 1] ในการเรียงจากน้อยไปมาก รอบแรกค้นหา 1 แล้วสลับมาไว้ตำแหน่งแรก จากนั้นพิจารณาเฉพาะส่วนที่เหลือ

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย
ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา
ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง
ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

จำนวนการดำเนินการขึ้นกับ n
สรุปเป็น Big O หลังตัดค่าคงที่และพจน์ที่มีผลน้อยกว่า

ประเด็นที่ควรตรวจสอบ

ระบุ n ว่าหมายถึงอะไร
ไม่ใช่ผลจากเวลาบนเครื่องเดียวเป็นคำอธิบายทั้งหมด
เปรียบเทียบกับอัลกอริทึมในบทอื่น
ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

3.4.1.2 ข้อดี ข้อจำกัด และสถานการณ์ใช้งานของ Selection Sort

ไม่มีอัลกอริทึมใดดีที่สุดทุกสถานการณ์ ต้องพิจารณาขนาดข้อมูล รูปแบบข้อมูล หน่วยความจำ และความต้องการรักษาลำดับเดิม
เหมาะสำหรับอธิบายการเลือกค่าต่ำสุด/สูงสุดและกรณีที่ต้องการลดจำนวนการสลับ

ตัวอย่างการเรียนรู้

เลือกวิธีเรียงที่เหมาะสมกับรายการขนาดเล็กเพื่อความเข้าใจ หรือเลือกวิธีที่โตช้ากว่าเมื่อข้อมูลมีจำนวนมาก

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
ถ้า len(ข้อมูล) < 2:  
    return ข้อมูล  
# เลือกอัลกอริทึมตามลักษณะงาน
```

ประเด็นที่ควรตรวจสอบ

อธิบายเหตุผลของการเลือกใช้

ระบุข้อจำกัดอย่างน้อยหนึ่งข้อ

แยกความง่ายในการเรียนรู้ออกจากประสิทธิภาพ

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

3.4.2 การเลือกใช้และการประยุกต์

ไม่มีวิธีใดที่ดีที่สุดทุกกรณี จึงต้องเลือกให้สัมพันธ์กับขนาดและชนิดของข้อมูล

3.4.2.1 โจทย์ฝึกประยุกต์ใช้ Selection Sort

โจทย์ประยุกต์ควรเปลี่ยนจากตัวเลขธรรมดาเป็นข้อมูลที่มีความหมาย เช่น คะแนน ราคา หรือเวลา แต่ยังใช้แกนหลักของอัลกอริทึมเดิม

ตัวอย่างการเรียนรู้

จัดอันดับคะแนนนักเรียนจากสูงไปต่ำ โดยเก็บรหัสนักเรียนไว้คู่กับคะแนนและไม่ทำให้ข้อมูลคู่แยกจากกัน

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def selection_sort(a):
    a = a.copy()
    for start in range(len(a)-1):
        min_i = start
        for i in range(start+1, len(a)):
            if a[i] < a[min_i]:
                min_i = i
        a[start], a[min_i] = a[min_i], a[start]
    return a
```

ประเด็นที่ควรตรวจสอบ

ระบุคีย์ที่ใช้เรียง

รักษาข้อมูลประกอบของสมาชิก

อธิบายเหตุผลของทิศทางการเรียง

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

3.4.2.2 สรุปทบทวนและคำถามตรวจสอบความเข้าใจเรื่อง Selection Sort

การสรุปควรตอบได้ว่าอัลกอริทึมเลือกหรือเปลี่ยนข้อมูลอย่างไร ใช้เวลาประมาณเท่าไร ใช้พื้นที่เพิ่มหรือไม่ และเหมาะกับงานลักษณะใด จากสรุปนี้สามารถสร้างคำถามแบบอธิบาย คำนวณ ติดตามผล และแก้ไขได้

ตัวอย่างการเรียนรู้

คำถามตัวอย่าง: ถ้าให้ข้อมูล [6, 3, 5, 1] อัลกอริทึมนี้ทำอะไรในรอบแรก และผลลัพธ์หลังจบรอบเป็นอย่างไร

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา
ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง
ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
คำถาม = 'อธิบายรอบแรกของ ' + ชื่ออัลกอริทึม  
คำตอบควรมีขั้นตอนและผลลัพธ์
```

ประเด็นที่ควรตรวจสอบ

คำตอบต้องอธิบายเหตุผล ไม่ใช่ตอบเฉพาะผลลัพธ์

เชื่อมโยงกับ Big O และข้อจำกัด

อ้างอิงหัวข้อย่อยที่ใช้ตอบได้

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

4 Insertion Sort

บทนี้อธิบายการสร้างส่วนที่เรียงแล้ว การเลือก Key การเลื่อนข้อมูล และการแทรกในตำแหน่งที่เหมาะสม

ผลลัพธ์การเรียนรู้ของบท

อธิบายแนวคิดด้วยภาษาของตนเองได้

ติดตามขั้นตอนจากตัวอย่างข้อมูลขนาดเล็กได้

อ่าน Pseudocode และโค้ดภาษา Python ได้

ตรวจสอบผลลัพธ์และอธิบายข้อผิดพลาดได้

4.1 แนวคิดพื้นฐานของ Insertion Sort

ส่วนนี้อธิบายกลไกหลักของ Insertion Sort และสถานะของข้อมูลระหว่างที่อัลกอริทึมทำงาน

4.1.1 กลไกการทำงาน

ผู้เรียนจะพิจารณาว่าอัลกอริทึมเลือก เปรียบเทียบ สลับ หรือเลื่อนสมาชิกอย่างไร

4.1.1.1 ความหมายและแนวคิดหลักของ Insertion Sort

Insertion Sort มองส่วนต้นของรายการว่าเรียงแล้ว แล้วหยิบสมาชิกถัดไปมาแทรกในตำแหน่งที่ถูกต้อง วิธีนี้คล้ายการจัดไฟในมือ เพราะผู้เล่นหยิบไพ่ที่ละใบแล้วเลื่อนใบที่ใหญ่กว่าออกไป

ตัวอย่างการเรียนรู้

ข้อมูล [5, 2, 4, 1] เริ่มจาก [5] เป็นส่วนเรียงแล้ว นำ 2 ไปแทรกด้านหน้า จากนั้นนำ 4 ไปแทรกระหว่าง 2 กับ 5

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
for i in range(1, n):
    key = a[i]
    j = i - 1
    while j >= 0 and a[j] > key:
        a[j + 1] = a[j]
        j -= 1
    a[j + 1] = key
```

ประเด็นที่ควรตรวจสอบ

อธิบายขั้นตอนโดยไม่พึ่งโค้ดได้

ระบุส่วนของข้อมูลที่เรียงแล้วและยังไม่เรียง

ใช้คำว่าเปรียบเทียบ สลับ หรือเลื่อนให้ถูกบริบท

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งชั้น

4.1.1.2 เจาะไขการเรียงจากน้อยไปมากด้วย Insertion Sort

การเรียงจากน้อยไปมากของ Insertion Sort ต้องทำให้ค่าที่เล็กกว่าอยู่ก่อนค่าที่ใหญ่กว่า โดยเลือกเจาะไขการเปรียบเทียบให้ตรงกับกลไกของอัลกอริทึม ในแต่ละรอบ

ตัวอย่างการเรียนรู้

ข้อมูล [5, 2, 4, 1] เริ่มจาก [5] เป็นส่วนเรียงแล้ว นำ 2 ไปแทรกด้านหน้า จากนั้นนำ 4 ไปแทรกระหว่าง 2 กับ 5

วิธีคิดทีละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละชั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def insertion_sort(a):
    a = a.copy()
    for i in range(1, len(a)):
        key = a[i]
        j = i - 1
        while j >= 0 and a[j] > key:
            a[j+1] = a[j]
            j -= 1
        a[j+1] = key
    return a
```

ประเด็นที่ควรตรวจสอบ

ตรวจค่าต่ำสุดและค่าสูงสุดหลังทำงาน

ทดสอบข้อมูลที่มีค่าซ้ำ

ตรวจว่าทิศทางผลลัพธ์ตรงกับโจทย์

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งชั้น

4.1.2 สถานะข้อมูลในแต่ละรอบ

การบันทึกสถานะก่อนและหลังแต่ละรอบทำให้เห็นว่าส่วนใดเรียงเสร็จแล้วและส่วนใดยังต้องตรวจต่อ

4.1.2.1 การทำงานทีละรอบของ Insertion Sort

การติดตามทีละรอบช่วยให้ผู้เรียนเห็นสถานะของรายการก่อนและหลังการดำเนินการหนึ่งครั้ง
ควรบันทึกค่าที่เปลี่ยน ตำแหน่งที่เปลี่ยน และส่วนที่ถือว่าเรียงเสร็จแล้ว ในแต่ละรอบ

ตัวอย่างการเรียนรู้

ข้อมูล [5, 2, 4, 1] เริ่มจาก [5] เป็นส่วนเรียงแล้ว นำ 2 ไปแทรกด้านหน้า จากนั้นนำ 4 ไปแทรกระหว่าง 2 กับ 5

วิธีคิดทีละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
for i in range(1, n):
    key = a[i]
    j = i - 1
    while j >= 0 and a[j] > key:
        a[j + 1] = a[j]
        j -= 1
    a[j + 1] = key
```

ประเด็นที่ควรตรวจสอบ

เขียนรายการหลังจบรอบทุกครั้ง

ระบุจำนวนรอบสูงสุด

อย่าข้ามการตรวจข้อมูลที่ยังไม่เรียง

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

4.1.2.2 การทำงานกับข้อมูลที่เรียงแล้วหรือมีค่าซ้ำ

ข้อมูลที่เรียงแล้วและข้อมูลที่มีค่าซ้ำเป็นกรณีทดสอบสำคัญ

เพราะช่วยตรวจว่าอัลกอริทึมหยุดได้ถูกต้องและไม่ทำลายลำดับที่ไม่จำเป็น ในแต่ละรอบ

ตัวอย่างการเรียนรู้

ข้อมูล [1, 2, 2, 3] ควรได้ผลลัพธ์เดิมและไม่เกิดการสลับที่เกินจำเป็น

วิธีคิดทีละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา
ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง
ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def insertion_sort(a):  
    a = a.copy()  
    for i in range(1, len(a)):  
        key = a[i]  
        j = i - 1  
        while j >= 0 and a[j] > key:  
            a[j+1] = a[j]  
            j -= 1  
        a[j+1] = key  
    return a
```

ประเด็นที่ควรตรวจสอบ

ทดสอบรายการว่างและรายการมีสมาชิกเดียว

ทดสอบค่าซ้ำทั้งหมด

พิจารณาการหยุดก่อนครบทุกขั้นเมื่อทำได้

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

4.2 ขั้นตอนการทำงานและการติดตามผลลัพธ์

ส่วนนี้แสดงการทำงานกับข้อมูลจริงตั้งแต่รอบแรกจนถึงผลลัพธ์สุดท้าย รวมถึงข้อมูลกรณีพิเศษ

4.2.1 การเรียงจากน้อยไปมาก

เริ่มจากทิศทางที่ใช้บ่อยที่สุด แล้วติดตามการเปลี่ยนแปลงของรายการในแต่ละรอบ

4.2.1.1 ตัวอย่างติดตามผลลัพธ์ของ Insertion Sort รอบที่ 1

การทำตารางติดตามช่วยเชื่อมแนวคิดกับผลลัพธ์จริง

ให้เขียนข้อมูลทุกครั้งที่มีการเปลี่ยนแปลงและทำเครื่องหมายส่วนที่เรียงเสร็จแล้ว

ตัวอย่างการเรียนรู้

ข้อมูล [5, 2, 4, 1] เริ่มจาก [5] เป็นส่วนเรียงแล้ว นำ 2 ไปแทรกด้านหน้า จากนั้นนำ 4 ไปแทรกระหว่าง 2 กับ 5

วิธีคิดทีละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
for i in range(1, n):
    key = a[i]
    j = i - 1
    while j >= 0 and a[j] > key:
        a[j + 1] = a[j]
        j -= 1
    a[j + 1] = key
```

ประเด็นที่ควรตรวจสอบ

บันทึกค่าก่อนเริ่มรอบ

บันทึกค่าหลังจบการดำเนินการ

ตรวจสอบว่าส่วนที่ประกาศว่าเรียงแล้วถูกต้อง

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

4.2.1.2 ตัวอย่างติดตามผลลัพธ์ของ Insertion Sort รอบสุดท้าย

รอบสุดท้ายควรทำให้เห็นว่ารายการครบถ้วนและไม่มีคู่ข้อมูลที่ผิดเงื่อนไข

การอธิบายรอบสุดท้ายช่วยให้สร้างคำถามตรวจสอบความเข้าใจได้

ตัวอย่างการเรียนรู้

ข้อมูล [5, 2, 4, 1] เริ่มจาก [5] เป็นส่วนเรียงแล้ว นำ 2 ไปแทรกด้านหน้า จากนั้นนำ 4 ไปแทรกระหว่าง 2 กับ 5

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุที่ต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def insertion_sort(a):
    a = a.copy()
    for i in range(1, len(a)):
        key = a[i]
        j = i - 1
        while j >= 0 and a[j] > key:
            a[j+1] = a[j]
            j -= 1
        a[j+1] = key
    return a
```

ประเด็นที่ควรตรวจสอบ

ตรวจสอบผลลัพธ์ด้วยการใส่คู่ข้อมูล

ตรวจสอบจำนวนสมาชิก

ตรวจสอบข้อมูลที่ซ้ำและค่าขอบเขต

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

4.2.2 รูปแบบข้อมูลพิเศษ

การเรียงจากมากไปน้อย ข้อมูลว่าง ข้อมูลซ้ำ และสมาชิกเดียวใช้ตรวจสอบคุณสมบัติของอัลกอริทึม

4.2.2.1 การดัดแปลงให้เรียงจากมากไปน้อยด้วย Insertion Sort

การเปลี่ยนทิศทางมักทำได้โดยกลับเงื่อนไขการเปรียบเทียบ หรือกลับผลลัพธ์หลังเรียง

วิธีที่เหมาะสมกับโครงสร้างอัลกอริทึมและเป้าหมายด้าน stability

ตัวอย่างการเรียนรู้

ข้อมูล [3, 1, 4, 2] ต้องได้ [4, 3, 2, 1] เมื่อกำหนด descending

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def insertion_sort(a):
    a = a.copy()
    for i in range(1, len(a)):
        key = a[i]
        j = i - 1
        while j >= 0 and a[j] > key:
            a[j+1] = a[j]
            j -= 1
        a[j+1] = key
    return a
```

ประเด็นที่ควรตรวจสอบ

เปลี่ยนเงื่อนไขอย่างเป็นระบบ

ทดสอบข้อมูลที่ซ้ำ

ระวังการกลับรายการหลังมีการแก้ไขข้อมูลต้นฉบับ

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

4.2.2.2 กรณีข้อมูลว่าง สมาชิกเดียว และสมาชิกซ้ำของ Insertion Sort

กรณีขอบเขตช่วยป้องกันข้อผิดพลาดจาก loop และ index รายการว่างควรคืนผลลัพธ์ว่าง

รายการที่มีสมาชิกเดียวไม่ต้องดำเนินการ และรายการที่มีสมาชิกซ้ำต้องยังคงจำนวนสมาชิกครบ

ตัวอย่างการเรียนรู้

[], [9] และ [2, 2, 1, 1] เป็นชุดข้อมูลทดสอบขั้นต่ำที่ควรมี

วิธีคิดทีละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def insertion_sort(a):
    a = a.copy()
    for i in range(1, len(a)):
        key = a[i]
        j = i - 1
        while j >= 0 and a[j] > key:
            a[j+1] = a[j]
            j -= 1
        a[j+1] = key
    return a
```

ประเด็นที่ควรตรวจสอบ

ไม่เข้าถึง index ของรายการว่าง

ไม่วนเกินขอบเขต

ตรวจสอบว่าค่าซ้ำไม่หาย

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

4.3 การเขียนอัลกอริทึมและโปรแกรม

จากแนวคิดจะถูกแปลงเป็น Pseudocode และ Python เพื่อให้ทดลองและตรวจสอบข้อผิดพลาดได้

4.3.1 Pseudocode และโค้ดภาษา Python

Pseudocode ช่วยตรวจสอบตรรกะ ส่วน Python ช่วยทดลองอัลกอริทึมกับข้อมูลขนาดเล็ก

4.3.1.1 Pseudocode ของ Insertion Sort

Pseudocode แสดงลำดับการทำงานโดยไม่ผูกกับภาษาโปรแกรมใดภาษาหนึ่ง

มีประโยชน์ต่อการตรวจสอบตรรกะก่อนเขียนโปรแกรมจริง ในแต่ละรอบ

ตัวอย่างการเรียนรู้

เริ่มจากรับรายการ กำหนดรอบหรือขอบเขต เปรียบเทียบข้อมูล ดำเนินการตามเงื่อนไข และคืนค่ารายการที่เรียงแล้ว

วิธีคิดทีละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
for i in range(1, n):
    key = a[i]
    j = i - 1
    while j >= 0 and a[j] > key:
        a[j + 1] = a[j]
        j -= 1
    a[j + 1] = key
```

ประเด็นที่ควรตรวจสอบ

กำหนดตัวแปรให้สื่อความหมาย

ระบุเงื่อนไขหยุด

แยกขั้นตอนหลักกับรายละเอียดการสลับหรือรวม

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

4.3.1.2 โค้ด Python ของ Insertion Sort

โค้ดภาษา Python ช่วยทดลองอัลกอริทึมด้วยข้อมูลขนาดเล็กได้ง่าย

การเขียนฟังก์ชันแยกจากส่วนรับข้อมูลทำให้ทดสอบซ้ำและนำไปสร้างคำถามได้สะดวก ในแต่ละรอบ

ตัวอย่างการเรียนรู้

ทดลองเรียง [5, 1, 4, 2] แล้วตรวจสอบด้วย assert ว่าผลลัพธ์เป็น [1, 2, 4, 5]

วิธีคิดทีละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def insertion_sort(a):  
    a = a.copy()  
    for i in range(1, len(a)):  
        key = a[i]  
        j = i - 1  
        while j >= 0 and a[j] > key:  
            a[j+1] = a[j]  
            j -= 1  
        a[j+1] = key  
    return a
```

ประเด็นที่ควรตรวจสอบ

ใช้ชื่อฟังก์ชันและตัวแปรชัดเจน

ทดสอบผลลัพธ์ด้วยข้อมูลหลายรูปแบบ

ไม่ใช่ sorted เป็นคำตอบแทนการฝึกเขียนอัลกอริทึม

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

4.3.2 การตรวจหาข้อผิดพลาดและการทดสอบ

การอ่านโค้ดและการทดสอบข้อมูลหลายรูปแบบช่วยยืนยันว่าผลลัพธ์ถูกต้องจริง

4.3.2.1 การอ่านโค้ดและค้นหาข้อผิดพลาดใน Insertion Sort

ผู้เรียนควรอ่านโค้ดโดยตามค่าของตัวแปรหลัก ไม่เดาจากชื่ออัลกอริทึมเพียงอย่างเดียว
ข้อผิดพลาดที่พบบ่อยคือขอบเขตลูป เงื่อนไขกลับด้าน และการไม่เก็บค่าก่อนสลับหรือเลื่อน

ตัวอย่างการเรียนรู้

ถ้าใช้ range ผิดหนึ่งตำแหน่ง อาจไม่ได้เปรียบเทียบสมาชิกคู่สุดท้ายหรือเกิด index เกินขอบเขต

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def insertion_sort(a):
    a = a.copy()
    for i in range(1, len(a)):
        key = a[i]
        j = i - 1
        while j >= 0 and a[j] > key:
            a[j+1] = a[j]
            j -= 1
        a[j+1] = key
    return a
```

ประเด็นที่ควรตรวจสอบ

ตรวจสอบจุดเริ่มและจุดสิ้นสุดของลูป

ตรวจสอบตัวแปรที่เปลี่ยนทุกครั้ง

สร้างข้อมูลเล็กเพื่อจำลองทีละบรรทัด

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

4.3.2.2 การทดสอบความถูกต้องของ Insertion Sort

การทดสอบความถูกต้องควรตรวจสอบเรื่องพร้อมกัน คือผลลัพธ์เรียงถูกต้องทิศทาง
และสมาชิกเดิมไม่หายหรือเพิ่มเกินมา

วิธีตรวจง่ายคือเปรียบเทียบกับผลลัพธ์ที่คาดหวังหรือใช้สำเนาข้อมูลต้นฉบับ

ตัวอย่างการเรียนรู้

หลังเรียง [4, 2, 4, 1] ให้ตรวจว่าได้ [1, 2, 4, 4] และจำนวนเลข 4 ยังเป็นสองตัว

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
ผล = อัลกอริทึม(ข้อมูล)
assert ผล == [1, 2, 4, 4]
```

ประเด็นที่ควรตรวจสอบ

ใช้ข้อมูลปกติและข้อมูลขอบเขต

ทดสอบทั้ง ascending และ descending

บันทึกผลการทดสอบเพื่อทำ Dataset

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

4.4 การวิเคราะห์และประยุกต์ใช้

ส่วนสุดท้ายเชื่อมโยงอัลกอริทึมกับประสิทธิภาพ ข้อจำกัด และสถานการณ์ใช้งาน

4.4.1 ประสิทธิภาพและความซับซ้อน

พิจารณาจำนวนการดำเนินการและสรุปแนวโน้มด้วย Big O ในระดับที่เหมาะสมกับปี 1

4.4.1.1 การวิเคราะห์เวลาและจำนวนการดำเนินการของ Insertion Sort

การวิเคราะห์ควรแยกการเปรียบเทียบ การสลับ การเลื่อน หรือการรวมตามลักษณะอัลกอริทึม แล้วสรุปเป็นอัตราการเติบโตที่เข้าใจง่าย ดีที่สุด $O(n)$ เมื่อข้อมูลเรียงหรือเกือบเรียง; กรณีแย่ที่สุด $O(n^2)$

ตัวอย่างการเรียนรู้

ข้อมูล [5, 2, 4, 1] เริ่มจาก [5] เป็นส่วนเรียงแล้ว นำ 2 ไปแทรกด้านหน้า จากนั้นนำ 4 ไปแทรกระหว่าง 2 กับ 5

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

จำนวนการดำเนินการขึ้นกับ n
สรุปเป็น Big O หลังตัดค่าคงที่และพจน์ที่มีผลน้อยกว่า

ประเด็นที่ควรตรวจสอบ

ระบุ n ว่าหมายถึงอะไร

ไม่ใช่ผลจากเวลาบนเครื่องเดียวเป็นคำอธิบายทั้งหมด

เปรียบเทียบกับอัลกอริทึมในบทอื่น

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

4.4.1.2 ข้อดี ข้อจำกัด และสถานการณ์ใช้งานของ Insertion Sort

ไม่มีอัลกอริทึมใดดีที่สุดทุกสถานการณ์ ต้องพิจารณาขนาดข้อมูล รูปแบบข้อมูล หน่วยความจำ และความต้องการรักษาลำดับเดิม เหมาะกับรายการขนาดเล็กหรือข้อมูลที่เกือบเรียงแล้ว และอธิบายการเลื่อนได้ชัดเจน

ตัวอย่างการเรียนรู้

เลือกวิธีเรียงที่เหมาะสมกับรายการขนาดเล็กเพื่อความเข้าใจ หรือเลือกวิธีที่โตช้ากว่าเมื่อข้อมูลมีจำนวนมาก

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
ถ้า len(ข้อมูล) < 2:  
    return ข้อมูล  
# เลือกอัลกอริทึมตามลักษณะงาน
```

ประเด็นที่ควรตรวจสอบ

อธิบายเหตุผลของการเลือกใช้

ระบุข้อจำกัดอย่างน้อยหนึ่งข้อ

แยกความง่ายในการเรียนรู้ออกจากประสิทธิภาพ

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

4.4.2 การเลือกใช้และการประยุกต์

ไม่มีวิธีใดที่ดีที่สุดทุกกรณี จึงต้องเลือกให้สัมพันธ์กับขนาดและชนิดของข้อมูล

4.4.2.1 โจทย์ฝึกประยุกต์ใช้ Insertion Sort

โจทย์ประยุกต์ควรเปลี่ยนจากตัวเลขธรรมดาเป็นข้อมูลที่มีความหมาย เช่น คะแนน ราคา หรือเวลา แต่ยังใช้แกนหลักของอัลกอริทึมเดิม

ตัวอย่างการเรียนรู้

จัดอันดับคะแนนนักเรียนจากสูงไปต่ำ โดยเก็บรหัสนักเรียนไว้คู่กับคะแนนและไม่ทำให้ข้อมูลคู่แยกจากกัน

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def insertion_sort(a):
    a = a.copy()
    for i in range(1, len(a)):
        key = a[i]
        j = i - 1
        while j >= 0 and a[j] > key:
            a[j+1] = a[j]
            j -= 1
        a[j+1] = key
    return a
```

ประเด็นที่ควรตรวจสอบ

ระบุคีย์ที่ใช้เรียง

รักษาข้อมูลประกอบของสมาชิก

อธิบายเหตุผลของทิศทางการเรียง

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

4.4.2.2 สรุปทบทวนและคำถามตรวจสอบความเข้าใจเรื่อง Insertion Sort

การสรุปควรตอบได้ว่าอัลกอริทึมเลือกหรือเปลี่ยนข้อมูลอย่างไร ใช้เวลาประมาณเท่าไร ใช้พื้นที่เพิ่มหรือไม่ และเหมาะกับงานลักษณะใด จากสรุปนี้สามารถสร้างคำถามแบบอธิบาย คำนวณ ติดตามผล และแก้โค้ดได้

ตัวอย่างการเรียนรู้

คำถามตัวอย่าง: ถ้าให้ข้อมูล [6, 3, 5, 1] อัลกอริทึมนี้ทำอะไรในรอบแรก และผลลัพธ์หลังจบรอบเป็นอย่างไร

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
คำถาม = 'อธิบายรอบแรกของ ' + ชื่ออัลกอริทึม
คำตอบควรมีขั้นตอนและผลลัพธ์
```

ประเด็นที่ควรตรวจสอบ

คำตอบต้องอธิบายเหตุผล ไม่ใช่ตอบเฉพาะผลลัพธ์

เชื่อมโยงกับ Big O และข้อจำกัด

อ้างอิงหัวข้อย่อยที่ใช้ตอบได้

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

5 Merge Sort

บทนี้แนะนำแนวคิด Divide and Conquer ในระดับเบื้องต้น
โดยเน้นการแบ่งข้อมูลและการผสานรายการที่เรียงแล้ว

ผลลัพธ์การเรียนรู้ของบท

- อธิบายแนวคิดด้วยภาษาของตนเองได้
- ติดตามขั้นตอนจากตัวอย่างข้อมูลขนาดเล็กได้
- อ่าน Pseudocode และโค้ดภาษา Python ได้
- ตรวจสอบผลลัพธ์และอธิบายข้อผิดพลาดได้

5.1 แนวคิดพื้นฐานของ Merge Sort

ส่วนนี้อธิบายกลไกหลักของ Merge Sort และสถานะของข้อมูลระหว่างที่อัลกอริทึมทำงาน

5.1.1 กลไกการทำงาน

ผู้เรียนจะพิจารณาว่าอัลกอริทึมเลือก เปรียบเทียบ สลับ หรือเลื่อนสมาชิกอย่างไร

5.1.1.1 ความหมายและแนวคิดหลักของ Merge Sort

Merge Sort ใช้แนวคิดแบ่งเพื่อเอาชนะ โดยแบ่งรายการออกเป็นส่วนเล็ก ๆ
จนแต่ละส่วนมีสมาชิกไม่เกินหนึ่งตัว แล้วผสานส่วนที่เรียงแล้วกลับเข้าด้วยกัน
การผสานจะเปรียบเทียบสมาชิกหน้าสุดของสองรายการย่อย

ตัวอย่างการเรียนรู้

ข้อมูล [5, 2, 4, 1] แบ่งเป็น [5, 2] และ [4, 1] จากนั้นแบ่งต่อเป็นรายการสมาชิกเดี่ยว แล้วผสาน [2, 5] กับ [1, 4] เป็น [1, 2, 4, 5]

วิธีคิดที่ละลำดับ

- อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย
- ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา
- ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง
- ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
merge_sort(a):  
    ถ้าขนาด a <= 1 ให้คืน a  
    แบ่ง a เป็นซ้ายและขวา  
    ซ้าย = merge_sort(ซ้าย)  
    ขวา = merge_sort(ขวา)  
    คืนค่า merge(ซ้าย, ขวา)
```

ประเด็นที่ควรตรวจสอบ

อธิบายขั้นตอนโดยไม่พึ่งโค้ดได้

ระบุส่วนของข้อมูลที่เรียงแล้วและยังไม่เรียง

ใช้คำว่าเปรียบเทียบ สลับ หรือเลื่อนให้ถูกบริบท

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งชั้น

5.1.1.2 เจาะลึกการเรียงจากน้อยไปมากด้วย Merge Sort

การเรียงจากน้อยไปมากของ Merge Sort ต้องทำให้ค่าที่เล็กกว่าอยู่ก่อนค่าที่ใหญ่กว่า

โดยเลือกเงื่อนไขการเปรียบเทียบให้ตรงกับกลไกของอัลกอริทึม ในแต่ละรอบ

ตัวอย่างการเรียนรู้

ข้อมูล [5, 2, 4, 1] แบ่งเป็น [5, 2] และ [4, 1] จากนั้นแบ่งต่อเป็นรายการสมาชิกเดียว แล้วผสาน [2, 5] กับ [1, 4] เป็น [1, 2, 4, 5]

วิธีคิดทีละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนีทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def merge_sort(a):
    if len(a) <= 1:
        return a
    mid = len(a)//2
    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])
    return merge(left, right)

def merge(left, right):
    out = []
    while left and right:
        out.append(left.pop(0) if left[0] <= right[0] else right.pop(0))
    return out + left + right
```

ประเด็นที่ควรตรวจสอบ

ตรวจค่าต่ำสุดและค่าสูงสุดหลังทำงาน

ทดสอบข้อมูลที่มีค่าซ้ำ

ตรวจว่าทิศทางผลลัพธ์ตรงกับโจทย์

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งชั้น

5.1.2 สถานะข้อมูลในแต่ละรอบ

การบันทึกสถานะก่อนและหลังแต่ละรอบทำให้เห็นว่าส่วนใดเรียงเสร็จแล้วและส่วนใดยังต้องตรวจต่อ

5.1.2.1 การทำงานทีละรอบของ Merge Sort

การติดตามทีละรอบช่วยให้ผู้เรียนเห็นสถานะของรายการก่อนและหลังการดำเนินการหนึ่งครั้ง
ควรบันทึกค่าที่เปลี่ยน ตำแหน่งที่เปลี่ยน และส่วนที่ถือว่าเรียงเสร็จแล้ว ในแต่ละรอบ

ตัวอย่างการเรียนรู้

ข้อมูล [5, 2, 4, 1] แบ่งเป็น [5, 2] และ [4, 1] จากนั้นแบ่งต่อเป็นรายการสมาชิกเดี่ยว แล้วผสาน [2, 5] กับ [1, 4] เป็น [1, 2, 4, 5]

วิธีคิดทีละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
merge_sort(a):  
    ถ้าขนาด a <= 1 ให้คืน a  
    แบ่ง a เป็นซ้ายและขวา  
    ซ้าย = merge_sort(ซ้าย)  
    ขวา = merge_sort(ขวา)  
    คืนค่า merge(ซ้าย, ขวา)
```

ประเด็นที่ควรตรวจสอบ

เขียนรายการหลังจบรอบทุกครั้ง

ระบุจำนวนรอบสูงสุด

อย่าข้ามการตรวจข้อมูลที่ยังไม่เรียง

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

5.1.2.2 การทำงานกับข้อมูลที่เรียงแล้วหรือมีค่าซ้ำ

ข้อมูลที่เรียงแล้วและข้อมูลที่มีค่าซ้ำเป็นกรณีทดสอบสำคัญ

เพราะช่วยตรวจว่าอัลกอริทึมหยุดได้ถูกต้องและไม่ทำลายลำดับที่ไม่จำเป็น ในแต่ละรอบ

ตัวอย่างการเรียนรู้

ข้อมูล [1, 2, 2, 3] ควรได้ผลลัพธ์เดิมและไม่เกิดการสลับที่เกินจำเป็น

วิธีคิดทีละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา
ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง
ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def merge_sort(a):
    if len(a) <= 1:
        return a
    mid = len(a)//2
    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])
    return merge(left, right)

def merge(left, right):
    out = []
    while left and right:
        out.append(left.pop(0) if left[0] <= right[0] else right.pop(0))
    return out + left + right
```

ประเด็นที่ควรตรวจสอบ

ทดสอบรายการว่างและรายการมีสมาชิกเดียว

ทดสอบค่าซ้ำทั้งหมด

พิจารณาการหยุดก่อนครบทุกขั้นเมื่อทำได้

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

5.2 ขั้นตอนการทำงานและการติดตามผลลัพธ์

ส่วนนี้แสดงการทำงานกับข้อมูลจริงตั้งแต่รอบแรกจนถึงผลลัพธ์สุดท้าย รวมถึงข้อมูลกรณีพิเศษ

5.2.1 การเรียงจากน้อยไปมาก

เริ่มจากทิศทางที่ใช้บ่อยที่สุด แล้วติดตามการเปลี่ยนแปลงของรายการในแต่ละรอบ

5.2.1.1 ตัวอย่างติดตามผลลัพธ์ของ Merge Sort รอบที่ 1

การทำตารางติดตามช่วยเชื่อมแนวคิดกับผลลัพธ์จริง

ให้เขียนข้อมูลทุกครั้งที่มีการเปลี่ยนแปลงและทำเครื่องหมายส่วนที่เรียงเสร็จแล้ว

ตัวอย่างการเรียนรู้

ข้อมูล [5, 2, 4, 1] แบ่งเป็น [5, 2] และ [4, 1] จากนั้นแบ่งต่อเป็นรายการสมาชิกเดียว แล้วผสาน [2, 5] กับ [1, 4] เป็น [1, 2, 4, 5]

วิธีคิดทีละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
merge_sort(a):  
    ถ้าขนาด a <= 1 ให้คืน a  
    แบ่ง a เป็นซ้ายและขวา  
    ซ้าย = merge_sort(ซ้าย)  
    ขวา = merge_sort(ขวา)  
    คืนค่า merge(ซ้าย, ขวา)
```

ประเด็นที่ควรตรวจสอบ

บันทึกค่าก่อนเริ่มรอบ

บันทึกค่าหลังจบการดำเนินการ

ตรวจสอบว่าส่วนที่ประกาศว่าเรียงแล้วถูกต้อง

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

5.2.1.2 ตัวอย่างติดตามผลลัพธ์ของ Merge Sort รอบสุดท้าย

รอบสุดท้ายควรทำให้เห็นว่ารายการครบถ้วนและไม่มีคู่ข้อมูลที่ผิดเงื่อนไข

การอธิบายรอบสุดท้ายช่วยให้สร้างคำถามตรวจสอบความเข้าใจได้

ตัวอย่างการเรียนรู้

ข้อมูล [5, 2, 4, 1] แบ่งเป็น [5, 2] และ [4, 1] จากนั้นแบ่งต่อเป็นรายการสมาชิกเดียว แล้วผสาน [2, 5] กับ [1, 4] เป็น [1, 2, 4, 5]

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def merge_sort(a):
    if len(a) <= 1:
        return a
    mid = len(a)//2
    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])
    return merge(left, right)

def merge(left, right):
    out = []
    while left and right:
        out.append(left.pop(0) if left[0] <= right[0] else right.pop(0))
    return out + left + right
```

ประเด็นที่ควรตรวจสอบ

ตรวจสอบผลลัพธ์ด้วยการใส่คู่ข้อมูล

ตรวจสอบจำนวนสมาชิก

ตรวจสอบข้อมูลที่ซ้ำและค่าขอบเขต

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

5.2.2 รูปแบบข้อมูลพิเศษ

การเรียงจากมากไปน้อย ข้อมูลว่าง ข้อมูลซ้ำ และสมาชิกเดียวใช้ตรวจสอบความสมบูรณ์ของอัลกอริทึม

5.2.2.1 การดัดแปลงให้เรียงจากมากไปน้อยด้วย Merge Sort

การเปลี่ยนทิศทางมักทำได้โดยกลับเงื่อนไขการเปรียบเทียบ หรือกลับผลลัพธ์หลังเรียง

วิธีที่เหมาะสมขึ้นกับโครงสร้างอัลกอริทึมและเป้าหมายด้าน stability

ตัวอย่างการเรียนรู้

ข้อมูล [3, 1, 4, 2] ต้องได้ [4, 3, 2, 1] เมื่อกำหนด descending

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def merge_sort(a):
    if len(a) <= 1:
        return a
    mid = len(a)//2
    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])
    return merge(left, right)

def merge(left, right):
    out = []
    while left and right:
        out.append(left.pop(0) if left[0] <= right[0] else right.pop(0))
    return out + left + right
```

ประเด็นที่ควรตรวจสอบ

เปลี่ยนเงื่อนไขอย่างเป็นระบบ

ทดสอบข้อมูลที่ซ้ำ

ระวังการกลับรายการหลังมีการแก้ไขข้อมูลต้นฉบับ

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

5.2.2.2 กรณีข้อมูลว่าง สมาชิกเดียว และสมาชิกซ้ำของ Merge Sort

กรณีขอบเขตช่วยป้องกันข้อผิดพลาดจากลูปและ index รายการว่างควรคืนผลลัพธ์ว่าง

รายการที่มีสมาชิกเดียวไม่ต้องดำเนินการ และรายการที่มีสมาชิกซ้ำต้องยังคงจำนวนสมาชิกครบ

ตัวอย่างการเรียนรู้

[], [9] และ [2, 2, 1, 1] เป็นชุดข้อมูลทดสอบขั้นต่ำที่ควรมี

วิธีคิดทีละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def merge_sort(a):
    if len(a) <= 1:
        return a
    mid = len(a)//2
    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])
    return merge(left, right)

def merge(left, right):
    out = []
    while left and right:
        out.append(left.pop(0) if left[0] <= right[0] else right.pop(0))
    return out + left + right
```

ประเด็นที่ควรตรวจสอบ

ไม่เข้าถึง index ของรายการว่าง

ไม่วนเกินขอบเขต

ตรวจสอบว่าค่าซ้ำไม่หาย

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

5.3 การเขียนอัลกอริทึมและโปรแกรม

จากแนวคิดจะถูกแปลงเป็น Pseudocode และ Python เพื่อให้ทดลองและตรวจสอบข้อผิดพลาดได้

5.3.1 Pseudocode และโค้ดภาษา Python

Pseudocode ช่วยตรวจสอบตรรกะ ส่วน Python ช่วยทดลองอัลกอริทึมกับข้อมูลขนาดเล็ก

5.3.1.1 Pseudocode ของ Merge Sort

Pseudocode แสดงลำดับการทำงานโดยไม่ผูกกับภาษาโปรแกรมใดภาษาหนึ่ง

มีประโยชน์ต่อการตรวจสอบตรรกะก่อนเขียนโปรแกรมจริง ในแต่ละรอบ

ตัวอย่างการเรียนรู้

เริ่มจากรับรายการ กำหนดรอบหรือขอบเขต เปรียบเทียบข้อมูล ดำเนินการตามเงื่อนไข และคืนค่ารายการที่เรียงแล้ว

วิธีคิดทีละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
merge_sort(a):  
    ถ้าขนาด a <= 1 ให้คืน a  
    แบ่ง a เป็นซ้ายและขวา  
    ซ้าย = merge_sort(ซ้าย)  
    ขวา = merge_sort(ขวา)  
    คืนค่า merge(ซ้าย, ขวา)
```

ประเด็นที่ควรตรวจสอบ

กำหนดตัวแปรให้สื่อความหมาย

ระบุเงื่อนไขหยุด

แยกขั้นตอนหลักกับรายละเอียดการสลับหรือรวม

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

5.3.1.2 โค้ด Python ของ Merge Sort

โค้ดภาษา Python ช่วยทดลองอัลกอริทึมด้วยข้อมูลขนาดเล็กได้ง่าย

การเขียนฟังก์ชันแยกจากส่วนรับข้อมูลทำให้ทดสอบซ้ำและนำไปสร้างคำถามได้สะดวก ในแต่ละรอบ

ตัวอย่างการเรียนรู้

ทดลองเรียง [5, 1, 4, 2] แล้วตรวจสอบด้วย assert ว่าผลลัพธ์เป็น [1, 2, 4, 5]

วิธีคิดทีละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def merge_sort(a):
    if len(a) <= 1:
        return a
    mid = len(a)//2
    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])
    return merge(left, right)

def merge(left, right):
    out = []
    while left and right:
        out.append(left.pop(0) if left[0] <= right[0] else right.pop(0))
    return out + left + right
```

ประเด็นที่ควรตรวจสอบ

ใช้ชื่อฟังก์ชันและตัวแปรชัดเจน

ทดสอบผลลัพธ์ด้วยข้อมูลหลายรูปแบบ

ไม่ใช่ sorted เป็นคำตอบแทนการฝึกเขียนอัลกอริทึม

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

5.3.2 การตรวจหาข้อผิดพลาดและการทดสอบ

การอ่านโค้ดและการทดสอบข้อมูลหลายรูปแบบช่วยยืนยันว่าผลลัพธ์ถูกต้องจริง

5.3.2.1 การอ่านโค้ดและค้นหาข้อผิดพลาดใน Merge Sort

ผู้เรียนควรอ่านโค้ดโดยตามค่าของตัวแปรหลัก ไม่เดาจากชื่ออัลกอริทึมเพียงอย่างเดียว
ข้อผิดพลาดที่พบบ่อยคือขอบเขตลูป เงื่อนไขกลับด้าน และการไม่เก็บค่าก่อนสลับหรือเลื่อน

ตัวอย่างการเรียนรู้

ถ้าใช้ range ผิดหนึ่งตำแหน่ง อาจไม่ได้เปรียบเทียบสมาชิกคู่สุดท้ายหรือเกิด index เกินขอบเขต

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def merge_sort(a):
    if len(a) <= 1:
        return a
    mid = len(a)//2
    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])
    return merge(left, right)

def merge(left, right):
    out = []
    while left and right:
        out.append(left.pop(0) if left[0] <= right[0] else right.pop(0))
    return out + left + right
```

ประเด็นที่ควรตรวจสอบ

ตรวจจุดเริ่มและจุดสิ้นสุดของลูป

ตรวจตัวแปรที่เปลี่ยนทุกครั้ง

สร้างข้อมูลเล็กเพื่อจำลองทีละบรรทัด

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

5.3.2.2 การทดสอบความถูกต้องของ Merge Sort

การทดสอบความถูกต้องควรตรวจสอบเรื่องพร้อมกัน คือผลลัพธ์เรียงถูกต้องทิศทาง
และสมาชิกเดิมไม่หายหรือเพิ่มเกินมา

วิธีตรวจง่ายคือเปรียบเทียบกับผลลัพธ์ที่คาดหวังหรือใช้สำเนาข้อมูลต้นฉบับ

ตัวอย่างการเรียนรู้

หลังเรียง [4, 2, 4, 1] ให้ตรวจว่าได้ [1, 2, 4, 4] และจำนวนเลข 4 ยังเป็นสองตัว

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
ผล = อัลกอริทึม(ข้อมูล)
assert ผล == [1, 2, 4, 4]
```

ประเด็นที่ควรตรวจสอบ

ใช้ข้อมูลปกติและข้อมูลขอบเขต

ทดสอบทั้ง ascending และ descending

บันทึกผลการทดสอบเพื่อทำ Dataset

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

5.4 การวิเคราะห์และประยุกต์ใช้

ส่วนสุดท้ายเชื่อมโยงอัลกอริทึมกับประสิทธิภาพ ข้อจำกัด และสถานการณ์ใช้งาน

5.4.1 ประสิทธิภาพและความซับซ้อน

พิจารณาจำนวนการดำเนินการและสรุปแนวโน้มด้วย Big O ในระดับที่เหมาะสมกับปี 1

5.4.1.1 การวิเคราะห์เวลาและจำนวนการดำเนินการของ Merge Sort

การวิเคราะห์ควรแยกการเปรียบเทียบ การสลับ การเลื่อน หรือการรวมตามลักษณะอัลกอริทึม แล้วสรุปเป็นอัตราการเติบโตที่เข้าใจง่าย $O(n \log n)$ ในกรณีทั่วไป และใช้พื้นที่ช่วยสำหรับการผสานในรูปแบบพื้นฐาน

ตัวอย่างการเรียนรู้

ข้อมูล [5, 2, 4, 1] แบ่งเป็น [5, 2] และ [4, 1] จากนั้นแบ่งต่อเป็นรายการสมาชิกเดี่ยว แล้วผสาน [2, 5] กับ [1, 4] เป็น [1, 2, 4, 5]

วิธีคิดที่ละเอียด

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย
ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา
ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง
ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

จำนวนการดำเนินการขึ้นกับ n
สรุปเป็น Big O หลังตัดค่าคงที่และพจน์ที่มีผลน้อยกว่า

ประเด็นที่ควรตรวจสอบ

ระบุ n ว่าหมายถึงอะไร
ไม่ใช่ผลจากเวลาบนเครื่องเดียวเป็นคำอธิบายทั้งหมด
เปรียบเทียบกับอัลกอริทึมในบทอื่น
ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

5.4.1.2 ข้อดี ข้อจำกัด และสถานการณ์ใช้งานของ Merge Sort

ไม่มีอัลกอริทึมใดที่ดีที่สุดทุกสถานการณ์ ต้องพิจารณาขนาดข้อมูล รูปแบบข้อมูล หน่วยความจำ และความต้องการรักษาลำดับเดิม เหมาะกับข้อมูลจำนวนมากเมื่อยอมใช้หน่วยความจำเพิ่ม และเป็นตัวอย่างพื้นฐานของ Divide and Conquer

ตัวอย่างการเรียนรู้

เลือกวิธีเรียงที่เหมาะสมกับรายการขนาดเล็กเพื่อความเข้าใจ หรือเลือกวิธีที่โตช้ากว่าเมื่อข้อมูลมีจำนวนมาก

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
ถ้า len(ข้อมูล) < 2:  
    return ข้อมูล  
# เลือกอัลกอริทึมตามลักษณะงาน
```

ประเด็นที่ควรตรวจสอบ

อธิบายเหตุผลของการเลือกใช้

ระบุข้อจำกัดอย่างน้อยหนึ่งข้อ

แยกความง่ายในการเรียนรู้ออกจากประสิทธิภาพ

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

5.4.2 การเลือกใช้และการประยุกต์

ไม่มีวิธีใดที่ดีที่สุดทุกกรณี จึงต้องเลือกให้สัมพันธ์กับขนาดและชนิดของข้อมูล

5.4.2.1 โจทย์ฝึกประยุกต์ใช้ Merge Sort

โจทย์ประยุกต์ควรเปลี่ยนจากตัวเลขธรรมดาเป็นข้อมูลที่มีความหมาย เช่น คะแนน ราคา หรือเวลา แต่ยังใช้แกนหลักของอัลกอริทึมเดิม

ตัวอย่างการเรียนรู้

จัดอันดับคะแนนนักเรียนจากสูงไปต่ำ โดยเก็บรหัสนักเรียนไว้คู่กับคะแนนและไม่ทำให้ข้อมูลคู่แยกจากกัน

วิธีคิดที่ละเอียด

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def merge_sort(a):
    if len(a) <= 1:
        return a
    mid = len(a)//2
    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])
    return merge(left, right)

def merge(left, right):
    out = []
    while left and right:
        out.append(left.pop(0) if left[0] <= right[0] else right.pop(0))
    return out + left + right
```

ประเด็นที่ควรตรวจสอบ

ระบุคีย์ที่ใช้เรียง

รักษาข้อมูลประกอบของสมาชิก

อธิบายเหตุผลของทิศทางการเรียง

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

5.4.2.2 สรุปทบทวนและคำถามตรวจสอบความเข้าใจเรื่อง Merge Sort

การสรุปควรตอบได้ว่าอัลกอริทึมเลือกหรือเปลี่ยนข้อมูลอย่างไร ใช้เวลาประมาณเท่าไร ใช้พื้นที่เพิ่มหรือไม่ และเหมาะกับงานลักษณะใด จากสรุปนี้สามารถสร้างคำถามแบบอธิบาย คำนวณ ติดตามผล และแก้โค้ดได้

ตัวอย่างการเรียนรู้

คำถามตัวอย่าง: ถ้าให้ข้อมูล [6, 3, 5, 1] อัลกอริทึมนี้ทำอะไรในรอบแรก และผลลัพธ์หลังจบรอบเป็นอย่างไร

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
คำถาม = 'อธิบายรอบแรกของ ' + ชื่ออัลกอริทึม
คำตอบควรมีขั้นตอนและผลลัพธ์
```

ประเด็นที่ควรตรวจสอบ

คำตอบต้องอธิบายเหตุผล ไม่ใช่ตอบเฉพาะผลลัพธ์

เชื่อมโยงกับ Big O และข้อจำกัด

อ้างอิงหัวข้อย่อยที่ใช้ตอบได้

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

6 Counting Sort

บทนี้อธิบายการนับความถี่ของจำนวนเต็ม การสร้างผลลัพธ์ และเงื่อนไขที่ทำให้ Counting Sort เหมาะหรือไม่เหมาะกับงาน

ผลลัพธ์การเรียนรู้ของบท

- อธิบายแนวคิดด้วยภาษาของตนเองได้
- ติดตามขั้นตอนจากตัวอย่างข้อมูลขนาดเล็กได้
- อ่าน Pseudocode และโค้ดภาษา Python ได้
- ตรวจสอบผลลัพธ์และอธิบายข้อผิดพลาดได้

6.1 แนวคิดพื้นฐานของ Counting Sort

ส่วนนี้อธิบายกลไกหลักของ Counting Sort และสถานะของข้อมูลระหว่างที่อัลกอริทึมทำงาน

6.1.1 กลไกการทำงาน

ผู้เรียนจะพิจารณาว่าอัลกอริทึมเลือก เปรียบเทียบ สลับ หรือเลื่อนสมาชิกอย่างไร

6.1.1.1 ความหมายและแนวคิดหลักของ Counting Sort

Counting Sort ไม่เปรียบเทียบสมาชิกเป็นคู่ แต่สร้างตารางนับจำนวนครั้งที่ค่าต่าง ๆ ปรากฏ แล้วสร้างผลลัพธ์ตามจำนวนที่นับ วิธีนี้เหมาะกับข้อมูลจำนวนเต็มที่ช่วงค่าหรือ k ไม่กว้างเกินไป

ตัวอย่างการเรียนรู้

ข้อมูล $[4, 2, 2, 1]$ สร้างตารางนับว่า 1 พบหนึ่งครั้ง 2 พบสองครั้ง และ 4 พบหนึ่งครั้ง จากนั้นอ่านตารางจากค่าต่ำไปสูง

วิธีคิดทีละลำดับ

- อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย
- ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา
- ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง
- ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
นับ = ตารางศูนย์
สำหรับค่า x ในข้อมูล:
    นับ[x] เพิ่มขึ้นหนึ่ง
สำหรับค่า x จากต่ำไปสูง:
    เขียน x ตามจำนวน นับ[x] ครั้ง
```

ประเด็นที่ควรตรวจสอบ

- อธิบายขั้นตอนโดยไม่พึ่งโค้ดได้

ระบุส่วนของข้อมูลที่เรียงแล้วและยังไม่เรียง

ใช้คำว่าเปรียบเทียบ สลับ หรือเลื่อนให้ถูกบริบท

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งชั้น

6.1.1.2 เจาะไขการเรียงจากน้อยไปมากด้วย Counting Sort

การเรียงจากน้อยไปมากของ Counting Sort ต้องทำให้ค่าที่เล็กกว่าอยู่ก่อนค่าที่ใหญ่กว่า โดยเลือกเจาะไขการเปรียบเทียบให้ตรงกับกลไกของอัลกอริทึม ในแต่ละรอบ

ตัวอย่างการเรียนรู้

ข้อมูล [4, 2, 2, 1] สร้างตารางนับว่า 1 พบหนึ่งครั้ง 2 พบสองครั้ง และ 4 พบหนึ่งครั้ง จากนั้นอ่านตารางจากค่าต่ำไปสูง

วิธีคิดทีละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละชั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def counting_sort(a):
    if not a:
        return []
    low, high = min(a), max(a)
    count = [0] * (high-low+1)
    for x in a:
        count[x-low] += 1
    out = []
    for i, c in enumerate(count):
        out.extend([i+low] * c)
    return out
```

ประเด็นที่ควรตรวจสอบ

ตรวจค่าต่ำสุดและค่าสูงสุดหลังทำงาน

ทดสอบข้อมูลที่มีค่าซ้ำ

ตรวจว่าทิศทางผลลัพธ์ตรงกับโจทย์

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งชั้น

6.1.2 สถานะข้อมูลในแต่ละรอบ

การบันทึกสถานะก่อนและหลังแต่ละรอบทำให้เห็นว่าส่วนใดเรียงเสร็จแล้วและส่วนใดยังต้องตรวจต่อ

6.1.2.1 การทำงานทีละรอบของ Counting Sort

การติดตามทีละรอบช่วยให้ผู้เรียนเห็นสถานะของรายการก่อนและหลังการดำเนินการหนึ่งครั้ง
ควรบันทึกค่าที่เปลี่ยน ตำแหน่งที่เปลี่ยน และส่วนที่ถือว่าเรียงเสร็จแล้ว ในแต่ละรอบ

ตัวอย่างการเรียนรู้

ข้อมูล [4, 2, 2, 1] สร้างตารางนับว่า 1 พบหนึ่งครั้ง 2 พบสองครั้ง และ 4 พบหนึ่งครั้ง
จากนั้นอ่านตารางจากค่าต่ำไปสูง

วิธีคิดทีละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
นับ = ตารางศูนย์
สำหรับค่า x ในข้อมูล:
    นับ[x] เพิ่มขึ้นหนึ่ง
สำหรับค่า x จากต่ำไปสูง:
    เขียน x ตามจำนวน นับ[x] ครั้ง
```

ประเด็นที่ควรตรวจสอบ

เขียนรายการหลังจบรอบทุกครั้ง

ระบุจำนวนรอบสูงสุด

อย่าข้ามการตรวจข้อมูลที่ยังไม่เรียง

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

6.1.2.2 การทำงานกับข้อมูลที่เรียงแล้วหรือมีค่าซ้ำ

ข้อมูลที่เรียงแล้วและข้อมูลที่มีค่าซ้ำเป็นกรณีทดสอบสำคัญ

เพราะช่วยตรวจว่าอัลกอริทึมหยุดได้ถูกต้องและไม่ทำลายลำดับที่ไม่จำเป็น ในแต่ละรอบ

ตัวอย่างการเรียนรู้

ข้อมูล [1, 2, 2, 3] ควรได้ผลลัพธ์เดิมและไม่เกิดการสลับที่เกินจำเป็น

วิธีคิดทีละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def counting_sort(a):  
    if not a:  
        return []  
    low, high = min(a), max(a)  
    count = [0] * (high-low+1)  
    for x in a:  
        count[x-low] += 1  
    out = []  
    for i, c in enumerate(count):  
        out.extend([i+low] * c)  
    return out
```

ประเด็นที่ควรตรวจสอบ

ทดสอบรายการว่างและรายการที่มีสมาชิกเดียว

ทดสอบค่าซ้ำทั้งหมด

พิจารณาการหยุดก่อนครบทุกขั้นเมื่อทำได้

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

6.2 ขั้นตอนการทำงานและการติดตามผลลัพธ์

ส่วนนี้แสดงการทำงานกับข้อมูลจริงตั้งแต่รอบแรกจนถึงผลลัพธ์สุดท้าย รวมถึงข้อมูลกรณีพิเศษ

6.2.1 การเรียงจากน้อยไปมาก

เริ่มจากทิศทางที่ใช้บ่อยที่สุด แล้วติดตามการเปลี่ยนแปลงของรายการในแต่ละรอบ

6.2.1.1 ตัวอย่างติดตามผลลัพธ์ของ Counting Sort รอบที่ 1

การทำตารางติดตามช่วยเชื่อมแนวคิดกับผลลัพธ์จริง

ให้เขียนข้อมูลทุกครั้งที่มีการเปลี่ยนแปลงและทำเครื่องหมายส่วนที่เรียงเสร็จแล้ว

ตัวอย่างการเรียนรู้

ข้อมูล [4, 2, 2, 1] สร้างตารางนับว่า 1 พบหนึ่งครั้ง 2 พบสองครั้ง และ 4 พบหนึ่งครั้ง
จากนั้นอ่านตารางจากค่าต่ำไปสูง

วิธีคิดทีละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
นับ = ตารางศูนย์
สำหรับค่า x ในข้อมูล:
    นับ[x] เพิ่มขึ้นหนึ่ง
สำหรับค่า x จากต่ำไปสูง:
    เขียน x ตามจำนวน นับ[x] ครั้ง
```

ประเด็นที่ควรตรวจสอบ

บันทึกค่าก่อนเริ่มรอบ

บันทึกค่าหลังจบการดำเนินการ

ตรวจสอบว่าส่วนที่ประกาศว่าเรียงแล้วถูกต้อง

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

6.2.1.2 ตัวอย่างติดตามผลลัพธ์ของ Counting Sort รอบสุดท้าย

รอบสุดท้ายควรทำให้เห็นว่ารายการครบถ้วนและไม่มีคู่ข้อมูลที่ผิดเงื่อนไข

การอธิบายรอบสุดท้ายช่วยให้สร้างคำถามตรวจสอบความเข้าใจได้

ตัวอย่างการเรียนรู้

ข้อมูล [4, 2, 2, 1] สร้างตารางนับว่า 1 พบหนึ่งครั้ง 2 พบสองครั้ง และ 4 พบหนึ่งครั้ง
จากนั้นอ่านตารางจากค่าต่ำไปสูง

วิธีคิดทีละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def counting_sort(a):
    if not a:
        return []
    low, high = min(a), max(a)
    count = [0] * (high-low+1)
    for x in a:
        count[x-low] += 1
    out = []
    for i, c in enumerate(count):
        out.extend([i+low] * c)
    return out
```

ประเด็นที่ควรตรวจสอบ

ตรวจสอบผลลัพธ์ด้วยการใส่คู่ข้อมูล

ตรวจสอบจำนวนสมาชิก

ตรวจสอบข้อมูลที่ซ้ำและค่าขอบเขต

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

6.2.2 รูปแบบข้อมูลพิเศษ

การเรียงจากมากไปน้อย ข้อมูลว่าง ข้อมูลซ้ำ และสมาชิกเดียวใช้ตรวจสอบคุณสมบัติของอัลกอริทึม

6.2.2.1 การดัดแปลงให้เรียงจากมากไปน้อยด้วย Counting Sort

การเปลี่ยนทิศทางมักทำได้โดยกลับเงื่อนไขการเปรียบเทียบ หรือกลับผลลัพธ์หลังเรียง

วิธีที่เหมาะสมกับโครงสร้างอัลกอริทึมและเป้าหมายด้าน stability

ตัวอย่างการเรียนรู้

ข้อมูล [3, 1, 4, 2] ต้องได้ [4, 3, 2, 1] เมื่อกำหนด descending

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def counting_sort(a):
    if not a:
        return []
    low, high = min(a), max(a)
    count = [0] * (high-low+1)
    for x in a:
        count[x-low] += 1
    out = []
    for i, c in enumerate(count):
        out.extend([i+low] * c)
    return out
```

ประเด็นที่ควรตรวจสอบ

เปลี่ยนเงื่อนไขอย่างเป็นระบบ

ทดสอบข้อมูลที่ซ้ำ

ระวังการกลับรายการหลังมีการแก้ข้อมูลต้นฉบับ

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

6.2.2.2 กรณีข้อมูลว่าง สมาชิกเดียว และสมาชิกซ้ำของ Counting Sort

กรณีขอบเขตช่วยป้องกันข้อผิดพลาดจากลูปและ index รายการว่างควรคืนผลลัพธ์ว่าง

รายการที่มีสมาชิกเดียวไม่ต้องดำเนินการ และรายการที่มีสมาชิกซ้ำต้องยังคงจำนวนสมาชิกครบ

ตัวอย่างการเรียนรู้

[], [9] และ [2, 2, 1, 1] เป็นชุดข้อมูลทดสอบขั้นต่ำที่ควรมี

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def counting_sort(a):
    if not a:
        return []
    low, high = min(a), max(a)
    count = [0] * (high-low+1)
    for x in a:
        count[x-low] += 1
    out = []
    for i, c in enumerate(count):
        out.extend([i+low] * c)
    return out
```

ประเด็นที่ควรตรวจสอบ

ไม่เข้าถึง index ของรายการว่าง

ไม่วนเกินขอบเขต

ตรวจสอบว่าค่าซ้ำไม่หาย

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

6.3 การเขียนอัลกอริทึมและโปรแกรม

จากแนวคิดจะถูกแปลงเป็น Pseudocode และ Python เพื่อให้ทดลองและตรวจสอบข้อผิดพลาดได้

6.3.1 Pseudocode และโค้ดภาษา Python

Pseudocode ช่วยตรวจสอบตรรกะ ส่วน Python ช่วยทดลองอัลกอริทึมกับข้อมูลขนาดเล็ก

6.3.1.1 Pseudocode ของ Counting Sort

Pseudocode แสดงลำดับการทำงานโดยไม่ผูกกับภาษาโปรแกรมใดภาษาหนึ่ง

มีประโยชน์ต่อการตรวจสอบตรรกะก่อนเขียนโปรแกรมจริง ในแต่ละรอบ

ตัวอย่างการเรียนรู้

เริ่มจากรับรายการ กำหนดรอบหรือขอบเขต เปรียบเทียบข้อมูล ดำเนินการตามเงื่อนไข และคืนค่ารายการที่เรียงแล้ว

วิธีคิดทีละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
นับ = ตารางศูนย์
สำหรับค่า x ในข้อมูล:
    นับ[x] เพิ่มขึ้นหนึ่ง
สำหรับค่า x จากต่ำไปสูง:
    เขียน x ตามจำนวน นับ[x] ครั้ง
```

ประเด็นที่ควรตรวจสอบ

กำหนดตัวแปรให้สื่อความหมาย

ระบุเงื่อนไขหยุด

แยกขั้นตอนหลักกับรายละเอียดการสลับหรือรวม

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

6.3.1.2 โค้ด Python ของ Counting Sort

โค้ดภาษา Python ช่วยทดลองอัลกอริทึมด้วยข้อมูลขนาดเล็กได้ง่าย

การเขียนฟังก์ชันแยกจากส่วนรับข้อมูลทำให้ทดสอบซ้ำและนำไปสร้างคำถามได้สะดวก ในแต่ละรอบ

ตัวอย่างการเรียนรู้

ทดลองเรียง [5, 1, 4, 2] แล้วตรวจสอบด้วย assert ว่าผลลัพธ์เป็น [1, 2, 4, 5]

วิธีคิดทีละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def counting_sort(a):
    if not a:
        return []
    low, high = min(a), max(a)
    count = [0] * (high-low+1)
    for x in a:
        count[x-low] += 1
    out = []
    for i, c in enumerate(count):
        out.extend([i+low] * c)
    return out
```

ประเด็นที่ควรตรวจสอบ

ใช้ชื่อฟังก์ชันและตัวแปรชัดเจน

ทดสอบผลลัพธ์ด้วยข้อมูลหลายรูปแบบ

ไม่ใช่ sorted เป็นคำตอบแทนการฝึกเขียนอัลกอริทึม

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

6.3.2 การตรวจหาข้อผิดพลาดและการทดสอบ

การอ่านโค้ดและการทดสอบข้อมูลหลายรูปแบบช่วยยืนยันว่าผลลัพธ์ถูกต้องจริง

6.3.2.1 การอ่านโค้ดและค้นหาข้อผิดพลาดใน Counting Sort

ผู้เรียนควรอ่านโค้ดโดยตามค่าของตัวแปรหลัก ไม่เดาจากชื่ออัลกอริทึมเพียงอย่างเดียว
ข้อผิดพลาดที่พบบ่อยคือขอบเขตสูงเกินไป ขาดการหวนซ้ำ และการไม่เก็บค่าก่อนสลับหรือเลื่อน

ตัวอย่างการเรียนรู้

ถ้าใช้ range ผิดหนึ่งตำแหน่ง อาจไม่ได้เปรียบเทียบสมาชิกคู่สุดท้ายหรือเกิด index เกินขอบเขต

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def counting_sort(a):
    if not a:
        return []
    low, high = min(a), max(a)
    count = [0] * (high-low+1)
    for x in a:
        count[x-low] += 1
    out = []
    for i, c in enumerate(count):
        out.extend([i+low] * c)
    return out
```

ประเด็นที่ควรตรวจสอบ

ตรวจสอบจุดเริ่มและจุดสิ้นสุดของลูป

ตรวจสอบตัวแปรที่เปลี่ยนทุกครั้ง

สร้างข้อมูลเล็กเพื่อจำลองทีละบรรทัด

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

6.3.2.2 การทดสอบความถูกต้องของ Counting Sort

การทดสอบความถูกต้องควรตรวจสอบเรื่องพร้อมกัน คือผลลัพธ์เรียงถูกต้องทิศทาง
และสมาชิกเดิมไม่หายหรือเพิ่มเกินมา

วิธีตรวจง่ายคือเปรียบเทียบกับผลลัพธ์ที่คาดหวังหรือใช้สำเนาข้อมูลต้นฉบับ

ตัวอย่างการเรียนรู้

หลังเรียง [4, 2, 4, 1] ให้ตรวจว่าได้ [1, 2, 4, 4] และจำนวนเลข 4 ยังเป็นสองตัว

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
ผล = อัลกอริทึม(ข้อมูล)
assert ผล == [1, 2, 4, 4]
```

ประเด็นที่ควรตรวจสอบ

ใช้ข้อมูลปกติและข้อมูลขอบเขต

ทดสอบทั้ง ascending และ descending

บันทึกผลการทดสอบเพื่อทำ Dataset

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

6.4 การวิเคราะห์และประยุกต์ใช้

ส่วนสุดท้ายเชื่อมโยงอัลกอริทึมกับประสิทธิภาพ ข้อจำกัด และสถานการณ์ใช้งาน

6.4.1 ประสิทธิภาพและความซับซ้อน

พิจารณาจำนวนการดำเนินการและสรุปแนวโน้มด้วย Big O ในระดับที่เหมาะสมกับปี 1

6.4.1.1 การวิเคราะห์เวลาและจำนวนการดำเนินการของ Counting Sort

การวิเคราะห์ควรแยกการเปรียบเทียบ การสลับ การเลื่อน หรือการรวมตามลักษณะอัลกอริทึม แล้วสรุปเป็นอัตราการเติบโตที่เข้าใจง่าย $O(n+k)$ เมื่อ n คือจำนวนข้อมูลและ k คือช่วงค่าที่ต้องสร้างตาราง; ใช้พื้นที่ $O(k)$

ตัวอย่างการเรียนรู้

ข้อมูล [4, 2, 2, 1] สร้างตารางนับว่า 1 พบหนึ่งครั้ง 2 พบสองครั้ง และ 4 พบหนึ่งครั้ง จากนั้นอ่านตารางจากค่าต่ำไปสูง

วิธีคิดที่ละเอียด

อ่านข้อมูลนำเข้าและระบุว่าจะต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

จำนวนการดำเนินการขึ้นกับ n
สรุปเป็น Big O หลังตัดค่าคงที่และพจน์ที่มีผลน้อยกว่า

ประเด็นที่ควรตรวจสอบ

ระบุ n ว่าหมายถึงอะไร

ไม่ใช่ผลจากเวลาบนเครื่องเดียวเป็นคำอธิบายทั้งหมด

เปรียบเทียบกับอัลกอริทึมในบทอื่น

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

6.4.1.2 ข้อดี ข้อจำกัด และสถานการณ์ใช้งานของ Counting Sort

ไม่มีอัลกอริทึมใดดีที่สุดทุกสถานการณ์ ต้องพิจารณาขนาดข้อมูล รูปแบบข้อมูล หน่วยความจำ และความต้องการรักษาลำดับเดิม เหมาะกับจำนวนเต็มที่ช่วงค่าไม่กว้างมาก แต่ไม่เหมาะเมื่อค่าสูงสุดกับค่าต่ำสุดห่างกันมาก

ตัวอย่างการเรียนรู้

เลือกวิธีเรียงที่เหมาะสมกับรายการขนาดเล็กเพื่อความเข้าใจ หรือเลือกวิธีที่โตช้ากว่าเมื่อข้อมูลมีจำนวนมาก

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
ถ้า len(ข้อมูล) < 2:  
    return ข้อมูล  
# เลือกอัลกอริทึมตามลักษณะงาน
```

ประเด็นที่ควรตรวจสอบ

อธิบายเหตุผลของการเลือกใช้

ระบุข้อจำกัดอย่างน้อยหนึ่งข้อ

แยกความง่ายในการเรียนรู้ออกจากประสิทธิภาพ

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

6.4.2 การเลือกใช้และการประยุกต์

ไม่มีวิธีใดที่ดีที่สุดทุกกรณี จึงต้องเลือกให้สัมพันธ์กับขนาดและชนิดของข้อมูล

6.4.2.1 โจทย์ฝึกประยุกต์ใช้ Counting Sort

โจทย์ประยุกต์ควรเปลี่ยนจากตัวเลขธรรมดาเป็นข้อมูลที่มีความหมาย เช่น คะแนน ราคา หรือเวลา แต่ยังใช้แกนหลักของอัลกอริทึมเดิม

ตัวอย่างการเรียนรู้

จัดอันดับคะแนนนักเรียนจากสูงไปต่ำ โดยเก็บรหัสนักเรียนไว้คู่กับคะแนนและไม่ทำให้ข้อมูลคู่แยกจากกัน

วิธีคิดที่ละเอียด

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
def counting_sort(a):
    if not a:
        return []
    low, high = min(a), max(a)
    count = [0] * (high-low+1)
    for x in a:
        count[x-low] += 1
    out = []
    for i, c in enumerate(count):
        out.extend([i+low] * c)
    return out
```

ประเด็นที่ควรตรวจสอบ

ระบุคีย์ที่ใช้เรียง

รักษาข้อมูลประกอบของสมาชิก

อธิบายเหตุผลของทิศทางการเรียง

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

6.4.2.2 สรุปทบทวนและคำถามตรวจสอบความเข้าใจเรื่อง Counting Sort

การสรุปควรตอบได้ว่าอัลกอริทึมเลือกหรือเปลี่ยนข้อมูลอย่างไร ใช้เวลาประมาณเท่าไร ใช้พื้นที่เพิ่มหรือไม่ และเหมาะกับงานลักษณะใด จากสรุปนี้สามารถสร้างคำถามแบบอธิบาย คำนวณ ติดตามผล และแก้โค้ดได้

ตัวอย่างการเรียนรู้

คำถามตัวอย่าง: ถ้าให้ข้อมูล [6, 3, 5, 1] อัลกอริทึมนี้ทำอะไรในรอบแรก และผลลัพธ์หลังจบรอบเป็นอย่างไร

วิธีคิดที่ละลำดับ

อ่านข้อมูลนำเข้าและระบุว่าต้องเรียงจากน้อยไปมากหรือมากไปน้อย

ระบุสมาชิก ตำแหน่ง หรือช่วงข้อมูลที่อัลกอริทึมกำลังพิจารณา

ทำตามเงื่อนไขของหัวข้อนี้ทีละขั้น แล้วบันทึกค่าที่เปลี่ยนแปลง

ตรวจสอบผลลัพธ์ว่าลำดับถูกต้องและสมาชิกเดิมยังอยู่ครบถ้วน

Pseudocode / โค้ดตัวอย่าง

```
คำถาม = 'อธิบายรอบแรกของ ' + ชื่ออัลกอริทึม
คำตอบควรมีขั้นตอนและผลลัพธ์
```

ประเด็นที่ควรตรวจสอบ

คำตอบต้องอธิบายเหตุผล ไม่ใช่ตอบเฉพาะผลลัพธ์

เชื่อมโยงกับ Big O และข้อจำกัด

อ้างอิงหัวข้อย่อยที่ใช้ตอบได้

ฝึก: อธิบายประเด็นหลักไม่เกิน 3 ประโยค และทดลองกับ [7, 3, 5, 1] หนึ่งขั้น

ตารางเปรียบเทียบอัลกอริทึมทั้ง 5 วิธี

ตารางนี้เป็นสรุปสำหรับทบทวน ไม่ควรใช้แทนคำอธิบายขั้นตอนในแต่ละบท

วิธี	แนวคิดหลัก	เวลาโดยทั่วไป	พื้นที่เพิ่ม	เหมาะกับ
Bubble Sort	เปรียบเทียบคู่ติดกันและสลับ	$O(n^2)$	น้อย	ฝึกแนวคิดพื้นฐาน
Selection Sort	เลือกค่าน้อยสุด/มากที่สุด	$O(n^2)$	น้อย	ลดจำนวนการสลับ
Insertion Sort	เลื่อนแล้วแทรก Key	$O(n^2)$	น้อย	ข้อมูลเล็กหรือเกือบเรียง
Merge Sort	แบ่งแล้วผสาน	$O(n \log n)$	เพิ่มตาม n	ข้อมูลจำนวนมาก
Counting Sort	นับความถี่ของค่าจำนวนเต็ม	$O(n+k)$	เพิ่มตาม k	ช่วงค่าไม่กว้าง

แนวทางเลือกใช้แบบง่าย: ถ้าต้องการสาธิตการสลับให้เริ่มจาก Bubble Sort; ถ้าต้องการลดจำนวนการสลับให้ดู Selection Sort; ถ้าข้อมูลเกือบเรียงแล้วให้พิจารณา Insertion Sort; ถ้าข้อมูลจำนวนมากให้ศึกษา Merge Sort; ถ้าข้อมูลเป็นจำนวนเต็มในช่วงแคบให้พิจารณา Counting Sort

แนวทางสร้างชุดคำถามประเมินแชทบอต 100 ข้อ

คำถามควรกระจายทั้งความจำ ความเข้าใจ การคำนวณ การติดตามขั้นตอน การอ่านโค้ด และการเลือกใช้งาน เพื่อไม่ให้ Dataset วัดเพียงการท่องจำ

บท	จำนวน	รูปแบบคำถามที่ควรมี
1 หลักการ	12	ความหมาย ทิศทาง เปรียบเทียบ Big O และ stability
2 Bubble	18	ติดตามรอบ สลับตำแหน่ง อ่านโค้ด และแก้เงื่อนไข
3 Selection	17	ค้นหาค่าต่ำสุด/สูงสุด จำนวนการสลับ และเลือกใช้
4 Insertion	18	Key การเลื่อน กรณีข้อมูลเกือบเรียง และอ่านลูป
5 Merge	18	แบ่งข้อมูล ผสานรายการ Recursion และ $O(n \log n)$
6 Counting	17	ตารางความถี่ ช่วงค่า $O(n+k)$ และข้อจำกัด
รวม	100	มีคำตอบอ้างอิงและหน้าที่มาของคำตอบทุกข้อ

รูปแบบคอลัมน์สำหรับไฟล์ Excel ที่จะทำในขั้นถัดไป: รหัสคำถาม | บท | หัวข้อย่อย | คำถาม | คำตอบอ้างอิง | ระดับความยาก | หน้าที่อ้างอิง | คำตอบจากแชทบอต | ผ่าน/ไม่ผ่าน |หมายเหตุผู้ประเมิน

อภิธานศัพท์ย่อ

อาร์เรย์ โครงสร้างข้อมูลที่เก็บสมาชิกตามตำแหน่ง index

คีย์ ค่าหรือคุณสมบัติที่ใช้เป็นเกณฑ์ในการจัดเรียง

การเปรียบเทียบ การตรวจสอบความสัมพันธ์ระหว่างสมาชิก เช่น มากกว่า น้อยกว่า หรือเท่ากัน

การสลับ การแลกเปลี่ยนตำแหน่งข้อมูลสองตำแหน่ง

การเลื่อน การขยับสมาชิกไปทิศทางเดียวเพื่อสร้างช่องว่าง

Stable Sort การเรียงที่รักษาลำดับเดิมของสมาชิกที่มีคีย์เท่ากัน

In-place การเรียงที่แก้ไขรายการเดิมโดยใช้พื้นที่เพิ่มเติมไม่มาก

Big O สัญลักษณ์อธิบายอัตราการเติบโตของงานเมื่อจำนวนข้อมูลเพิ่ม

Divide and Conquer การแบ่งปัญหาเป็นส่วนย่อย แก้ส่วนย่อย แล้วรวมคำตอบ

ความถี่ จำนวนครั้งที่ค่าหนึ่งปรากฏในข้อมูล

เอกสารอ้างอิงภาษาไทย

1. จุฬาลงกรณ์มหาวิทยาลัย, ภาควิชาวิศวกรรมคอมพิวเตอร์. (ม.ป.ป.). บทที่ อาร์เรย์และการจัดเรียงข้อมูล. <https://www.cp.eng.chula.ac.th/~vishnu/datastructure/aj.vishnu%20-%20arrayAndSorting.pdf>
2. จุฬาลงกรณ์มหาวิทยาลัย. (ม.ป.ป.). การเรียงลำดับข้อมูล: ULearn Data Structures. https://www.cp.eng.chula.ac.th/~somchai/ULearn/DataStructures/Topics/index_ch14.htm
3. มหาวิทยาลัยเทคโนโลยีราชมงคลล้านนา. (2567). รายวิชาโครงสร้างข้อมูลและอัลกอริทึม: คำอธิบายรายวิชาและจุดมุ่งหมาย. <https://lms.rmutl.ac.th/tqf/detail/28834716631177281/65ad4d20cadce9d12935fc195d046576>
4. สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง. (ม.ป.ป.). 06016316 โครงสร้างข้อมูลและอัลกอริทึม. <https://www.it.kmitl.ac.th/th/subjects/06016316-data-structures-and-algorithms>
5. มหาวิทยาลัยธรรมศาสตร์. (2561). รายละเอียดหลักสูตรวิศวกรรมคอมพิวเตอร์และรายวิชาที่เกี่ยวข้องกับการเรียงลำดับข้อมูล. <https://web2.reg.tu.ac.th/th/Picture/AttFile/d82b72c2-9ae3-4330-a3c6-ce1b40e99eb9>

หมายเหตุ: เนื้อหาในเอกสารนี้เป็นการเรียบเรียงใหม่เพื่อการเรียนรู้ระดับมหาวิทยาลัยปี 1 ไม่ได้คัดลอกข้อความจากแหล่งอ้างอิงแบบคำต่อคำ และควรตรวจสอบ URL อีกครั้งก่อนนำไปใช้อ้างอิงในรายงานทางวิชาการ